Mini Project Report On

LIVING LIBRARY: A SEMANTIC MEMORY SYSTEM FOR EFFICIENT KNOWLEDGE STORAGE

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science & Engineering

By

Muhammed Yazid Zain (U2303162)

Nikitha Praveen Basil (U2303171)

Niya Elizabeth Tomy (U2303173)

Rebecca Oommen (U2303186)

Under the guidance of
Ms. Janani K

Department of Computer Science & Engineering
Rajagiri School of Engineering and Technology (Autonomous)
(Affiliated to APJ Abdul Kalam Technological University)
Rajagiri Valley, Kakkanad, Kochi, 682039

May 2026

CERTIFICATE

This is to certify that the mini project report entitled ***"Living Library: A Semantic Memory System for Efficient Knowledge Storage"*** *is a bonafide record of the work done by* ***Muhammed Yazid Zain (U2303162)***, ***Nikitha Praveen Basil (U2303171)***, ***Niya Elizabeth Tomy (U2303173)***, ***Rebecca Oommen (U2303186)***, *submitted to the APJ Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B. Tech.) in Computer Science and Engineering during the academic year 2025-2026.*

Ms. Janani K
Project Guide
Asst. Professor
Dept. of CSE
RSET

Ms. Mehbooba P Shareef
Project Coordinator
Asst. Professor
Dept. of CSE
RSET

Dr. Preetha K.G
Professor & HoD
Dept. of CSE
RSET

ACKNOWLEDGEMENTS

We wish to express our sincere gratitude towards **Rev. Dr. Jaison Paul Mulerickal CMI**, Principal of RSET, and **Dr. Preetha K.G.**, Head of the Department of Computer Science and Engineering for providing us with the opportunity to undertake our mini project, "Living Library: A Semantic Memory System for Efficient Knowledge Storage".

We are highly indebted to our project coordinator, **Ms. Mehbooba P. Shareef**, Asst. Professor, Department of Computer Science and Engineering for their valuable support.

It is indeed our pleasure and a moment of satisfaction for us to express our sincere gratitude to our project guide **Ms. Janani K** for her patience and all the priceless advice and wisdom she has shared with us.

Last but not the least, We would like to express our sincere gratitude towards all other teachers and friends for their continuous support and constructive ideas.

Muhammed Yazid Zain
Nikitha Praveen Basil
Niya Elizabeth Tomy
Rebecca Oommen

Abstract

Modern artificial intelligence systems commonly store and retrieve large volumes of raw text, which leads to increased storage requirements, slower retrieval processes, and a higher likelihood of inaccurate or inconsistent outputs. The Living Library project proposes a semantic memory framework designed to improve knowledge storage efficiency and reliability. Instead of storing entire documents, the system extracts essential semantic facts and compresses them into structured concept-based memory units called concept capsules. These capsules are converted into numerical vector representations using Sentence-BERT, enabling efficient semantic similarity operations. The system employs a hybrid storage approach that combines vector-based semantic retrieval with structured representation of relationships between concepts. When a user query is received, the system reconstructs explanations from stored semantic knowledge rather than retrieving raw text. An integrated Audit Loop verifies generated outputs against stored semantic facts, reducing the risk of inconsistencies and semantic drift. Compared to traditional raw text storage approaches, the proposed system improves storage efficiency, supports adaptive explanation generation, and enhances the reliability of AI-generated knowledge retrieval.

Contents

Acknowledgements **i**

Abstract **ii**

List of Figures **vi**

List of Tables **vii**

List of Abbreviations **viii**

1 Introduction **1**

1.1 Background 1

1.2 Problem Definition 1

1.3 Scope and Motivation 2

1.4 Objectives 2

1.5 Challenges 3

1.6 Assumptions 3

1.7 Societal / Industrial Relevance 4

1.8 Organization of the Report 4

2 Software Requirements Specification **6**

2.1 Introduction 6

2.2 Overall Description 6

2.3 External Interface Requirements 7

2.3.1 User Interface 7

2.3.2 Software Interface 7

2.3.3 Hardware Interface 7

2.3.4 Communication Interface 8

2.4 System Features 8

2.4.1 Semantic Knowledge Storage 8
2.4.2 Concept Capsule Memory 8
2.4.3 Hybrid Storage 8
2.4.4 Adaptive Explanation Generation 9
2.4.5 Audit Verification 9
2.4.6 Semantic Drift Detection 9
2.4.7 Memory Compression 9
2.4.8 Knowledge Graph Visualization 9
2.4.9 Confidence and Truth Score 9
2.5 Other Nonfunctional Requirements 10
2.5.1 Performance Requirements 10
2.5.2 Reliability Requirements 10
2.5.3 Storage Efficiency 10
2.5.4 Scalability 10
2.5.5 Usability 10
2.5.6 Maintainability 10
2.5.7 Security 10
2.5.8 Portability 11
2.5.9 Accuracy Requirements 11

3 System Architecture and Design 12

3.1 System Overview 12
3.2 Architectural Design 13
3.3 Dataset identified 15
3.4 Proposed Methodology/Algorithms 16
3.4.1 Algorithm 1: Knowledge Extraction Process 16
3.4.2 Algorithm 2: Query Processing and Retrieval 17
3.5 User Interface Design 17
3.6 Database Design 17
3.7 Description of Implementation Strategies 18
3.8 Module Division 19
3.8.1 Text Ingestion Module 19

3.8.2 Knowledge Extraction Module 19
3.8.3 Embedding and Compression Module 19
3.8.4 Knowledge Storage Module 19
3.8.5 Query Processing and Retrieval Module 20
3.8.6 Audit and Verification Module 20
3.8.7 Module Allocation 20
3.8.8 Work Schedule – Gantt Chart 21

4 Results and Discussions **22**
4.1 Overview 22
4.2 Testing 22
4.3 Quantitative Results 25
4.4 Graphical Analysis 25
4.5 Discussion 28

5 Conclusion **29**
5.1 Conclusion 29
5.2 Future Scope 30

References **31**

Appendix A: Presentation **32**

Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes **55**

Appendix C: CO-PO-PSO Mapping **61**

List of Figures

3.1 Sequence Diagram – Query Processing and Response Generation 13

3.2 System Architecture of the Living Library Semantic Memory System . . . 14

3.3 Data Flow Diagram of the Living Library System 15

3.4 Project Work Schedule for the Living Library System 21

4.1 System response showing ingestion of input text, extraction of triples, query-based retrieval, and performance metrics such as compression and accuracy. 23

4.2 Audit Lab interface displaying fact-checking of extracted triples along with their verification status. 23

4.3 Semantic drift analysis, compression efficiency, and truth score stability visualizations showing system performance. 24

4.4 Knowledge graph and provenance flow representing extracted triples and their transformation into structured knowledge. 24

4.5 Performance metrics showing truth score and accuracy of the system. . . . 26

4.6 Comparison of original and compressed data sizes. 27

4.7 Compression distribution showing percentage of data reduced and retained after processing. 27

List of Tables

3.1 Module Contribution Distribution 20

4.1 Quantitative performance metrics of the system 25

List of Abbreviations

Acronym	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
CLI	Command Line Interface
DB	Database
LLM	Large Language Model
NLP	Natural Language Processing
SBERT	Sentence Bidirectional Encoder Representations from Transformers
SRS	Software Requirements Specification
UI	User Interface

Chapter 1

Introduction

1.1 Background

As digital information continues to increase, managing large collections of textual data has become increasingly difficult. Traditional databases and search engines usually rely on keyword-based retrieval, where documents are returned based on matching words or phrases. While this approach works in many cases, it does not always capture the full meaning of the text or the relationships between different concepts.

Recent developments in natural language processing have introduced techniques that allow computers to understand text in a more meaningful way. Methods such as semantic embeddings represent sentences as numerical vectors that capture their meaning, allowing systems to compare and retrieve information based on semantic similarity rather than just keywords.

In addition, knowledge graphs have become an important way of representing structured information. In a knowledge graph, entities and their relationships are stored as connected nodes, making it easier to understand how different pieces of information are related.

The Living Library project builds on these ideas by combining semantic embeddings with knowledge graph representations and memory compression techniques. This combination allows the system to store information in a structured and compact form while still preserving its meaning.

1.2 Problem Definition

Many existing retrieval systems focus mainly on returning documents that are similar to a user's query. While this can be useful, it often results in retrieving large sections of text that contain redundant or partially relevant information. In many cases, the actual

relationships or facts within the text are not explicitly stored or organized.

Another limitation is that storing entire documents requires significant memory and computational resources. Large textual contexts can become difficult to manage, especially when the system needs to search through them repeatedly to find specific information.

Because of these limitations, there is a need for a system that can transform unstructured text into structured knowledge representations. Such a system should be able to store information efficiently, preserve meaningful relationships between concepts, and retrieve relevant knowledge when required.

1.3 Scope and Motivation

The Living Library project focuses on developing a system that converts textual information into structured knowledge and stores it in a way that allows efficient retrieval. The system extracts relationships from text, compresses them into compact semantic representations, and stores them using a combination of vector storage and graph-based structures.

The motivation for this work comes from the growing need for intelligent information systems that can organize knowledge more effectively. Instead of relying only on keyword matching or large context storage, the Living Library approach attempts to capture the core meaning of information and represent it in a structured format.

By reducing redundancy and preserving important relationships between concepts, the system aims to improve both the efficiency and reliability of knowledge retrieval. This approach can also make it easier to understand how different pieces of information are connected.

1.4 Objectives

- To convert unstructured textual data into structured knowledge using relationship extraction techniques.
- To compress extracted knowledge into compact semantic representations while preserving meaning.
- To generate semantic embeddings that allow efficient similarity-based retrieval.

- To store knowledge using a hybrid architecture combining vector databases and graph-based structures.
- To implement mechanisms for auditing and evaluating the reliability of stored knowledge.
- To reconstruct meaningful answers from the stored knowledge when queries are made.

1.5 Challenges

Developing the Living Library system involved several challenges. One major challenge was extracting accurate relationships from natural language text. Sentences often contain complex structures, multiple clauses, and contextual meanings that can make automated extraction difficult.

Another challenge was ensuring that the compression process reduces redundancy without losing important information. If the compression is too aggressive, some useful details may be lost, which can affect the quality of retrieved answers.

Integrating different components of the system also required careful coordination. The system involves multiple stages including text ingestion, relationship extraction, embedding generation, and storage in both vector and graph databases. Ensuring smooth communication between these components was an important part of the design process.

Finally, verifying the reliability of stored knowledge was another challenge. Mechanisms were needed to check whether extracted information remains consistent and accurate over time.

1.6 Assumptions

- The input documents provided to the system contain factually correct and well-structured textual information.
- The extracted semantic triples accurately represent the core meaning of the original text.

- The compression of knowledge into concept capsules does not significantly affect the semantic integrity of the information.
- Sentence-BERT embeddings are assumed to effectively capture semantic similarity between concepts.
- The prototype operates on text-based data only, without handling multimedia inputs such as images, audio, or video.
- The system is designed for experimental and research purposes, running on standard computing hardware with moderate data sizes.
- The audit loop verification mechanism is assumed to be sufficient to detect major inconsistencies in generated explanations.

1.7 Societal / Industrial Relevance

Efficient knowledge management is important in many fields, including research, education, and industry. Organizations often work with large collections of textual information such as technical documents, research articles, and internal knowledge bases. Retrieving relevant knowledge from these large datasets can be time-consuming and inefficient.

The Living Library approach offers a way to organize information more effectively by converting text into structured semantic knowledge. This can help reduce redundancy, improve retrieval accuracy, and provide clearer insights into how different pieces of information are connected.

Such systems can be useful in applications such as digital libraries, research knowledge bases, enterprise documentation systems, and intelligent information retrieval platforms where structured and interpretable knowledge representation is valuable.

1.8 Organization of the Report

This report is organized into three chapters. Chapter 1 presents the introduction to the project, including the background, problem definition, objectives, challenges, assumptions, and the societal or industrial relevance of the proposed system.

Chapter 2 describes the Software Requirements Specification (SRS) of the system. It includes the overall description of the system, external interface requirements, system features, and non-functional requirements such as performance, reliability, usability, scalability, and security.

Chapter 3 explains the system architecture and design of the proposed Living Library system. It includes the architectural design, dataset identification, proposed methodology, user interface design, database design, implementation strategies, and module division of the system.

Chapter 2

Software Requirements Specification

2.1 Introduction

The Living Library system is a semantic memory framework designed to store and retrieve knowledge efficiently by extracting core facts from documents instead of storing full text. Traditional AI systems store raw text, which increases memory usage and slows down search operations. This system converts documents into structured semantic triples and stores them in compressed concept capsules, allowing efficient knowledge storage and reconstruction of explanations when required. The project belongs to the domain of Artificial Intelligence and Natural Language Processing and includes an audit mechanism to verify factual correctness and reduce semantic drift.

2.2 Overall Description

The Living Library system processes text documents by extracting meaningful information using Natural Language Processing techniques. The extracted information is converted into subject–relation–object triples and stored in a hybrid storage system consisting of a graph database and a vector database. The graph database stores relationships between concepts, while the vector database stores semantic embeddings for similarity search. When a user asks a question, the system retrieves relevant triples, reconstructs the answer, and verifies it using stored knowledge.

The scope of the project includes semantic knowledge extraction, memory compression, adaptive explanation generation, and audit-based verification of answers. The system supports only text-based documents such as TXT, PDF, and DOCX files. It does not include real-time learning, speech processing, or large-scale chatbot deployment. The main goal of the system is to demonstrate efficient AI memory storage using concept

capsules while maintaining accuracy, low semantic drift, and high storage efficiency.

2.3 External Interface Requirements

The system provides a web-based interface that allows users to upload documents and ask questions. The frontend communicates with the backend using REST API requests, and the backend processes the request using NLP and database operations.

2.3.1 User Interface

- File upload option for TXT, PDF, and DOCX files
- Chat interface for asking questions
- Dashboard showing compression, accuracy, and truth score
- Graph visualization of stored knowledge

2.3.2 Software Interface

- Python Flask backend for API handling
- spaCy for natural language processing
- SentenceTransformer for semantic similarity
- Neo4j for graph storage
- ChromaDB for vector storage
- PyPDF2 and python-docx for file reading

2.3.3 Hardware Interface

- Standard personal computer or laptop
- Minimum 8 GB RAM
- Intel i5 or equivalent processor
- Minimum 10 GB free disk space

2.3.4 Communication Interface

- HTTP requests between frontend and backend
- JSON data format for request and response
- Local database connection for Neo4j and ChromaDB

Example compression equation:

$$CompressionRatio = \frac{ParsedTextSize}{CompressedSize}$$

$$StorageSaved = \frac{Original - Compressed}{Original} \times 100$$

2.4 System Features

2.4.1 Semantic Knowledge Storage

The system extracts important facts from the uploaded text documents and stores them in the form of subject–relation–object triples. This allows the knowledge to be stored in a structured format instead of raw text.

2.4.2 Concept Capsule Memory

The extracted triples are grouped into compressed memory units called concept capsules. These capsules store only the essential semantic information, which helps reduce memory usage while keeping the meaning intact.

2.4.3 Hybrid Storage

The system uses a hybrid storage approach consisting of a graph database and a vector database. The graph database stores relationships between concepts, while the vector database stores semantic embeddings for similarity-based retrieval.

2.4.4 Adaptive Explanation Generation

The system generates answers dynamically from stored knowledge instead of copying text directly from the document. This allows the system to reconstruct explanations based on the stored triples.

2.4.5 Audit Verification

All generated answers are verified using the stored knowledge triples. This verification step ensures that the response is based only on the stored facts and helps prevent incorrect outputs.

2.4.6 Semantic Drift Detection

The system measures the similarity between the stored facts and the generated answers. If the difference is high, the system detects semantic drift and reduces the reliability score.

2.4.7 Memory Compression

Instead of storing complete documents, the system stores only extracted triples. This reduces storage size and improves efficiency while maintaining the important information.

2.4.8 Knowledge Graph Visualization

The stored triples are also represented as a knowledge graph where concepts are shown as nodes and relationships are shown as edges. This helps visualize the stored knowledge clearly.

2.4.9 Confidence and Truth Score

Each generated answer is assigned a confidence score and truth score based on semantic similarity and verification results. These values indicate how reliable the generated answer is.

2.5 Other Nonfunctional Requirements

2.5.1 Performance Requirements

The system should respond to user queries within a few seconds. Triple extraction and storage operations should complete within a reasonable amount of time for small and medium sized documents.

2.5.2 Reliability Requirements

The system must generate answers only from the stored knowledge base. The audit verification mechanism should ensure that incorrect or unsupported outputs are not produced.

2.5.3 Storage Efficiency

The system should reduce memory usage by storing extracted triples instead of full documents. The compression ratio should show improvement compared to raw text storage.

2.5.4 Scalability

The system should support multiple documents without performance degradation. The databases used for storage should be capable of handling a large number of triples.

2.5.5 Usability

The user interface should be simple and easy to understand. Users should be able to upload documents and ask questions without requiring technical knowledge.

2.5.6 Maintainability

The code should be modular so that each component such as extraction, storage, and query processing can be modified independently without affecting the entire system.

2.5.7 Security

The system should use only the documents uploaded by the user for answering queries. External data should not be accessed automatically to ensure safe operation.

2.5.8 Portability

The system should be able to run on different operating systems such as Windows, Linux, and macOS without requiring major changes.

2.5.9 Accuracy Requirements

The system should maintain a high truth score during verification. Semantic drift between stored facts and generated answers should remain low.

Chapter 3

System Architecture and Design

3.1 System Overview

The Living Library system is designed as a semantic memory framework that stores and retrieves knowledge in a structured and compressed form. Traditional information systems often store large amounts of raw text, which makes retrieval inefficient and context understanding difficult. The Living Library addresses this issue by transforming unstructured textual information into structured semantic representations.

The overall workflow of the system begins with the ingestion of textual data. Input documents are read and processed using natural language processing techniques to segment the text into sentences. Each sentence is then analyzed to extract semantic relationships represented as subject–predicate–object triples. These triples capture the meaning of sentences in a structured format that can be stored and queried efficiently.

Once the triples are extracted, the system performs semantic compression to reduce redundancy. Related triples are grouped together into concept capsules. This compression mechanism helps reduce storage requirements while preserving meaningful relationships between concepts. The compressed knowledge units are then converted into semantic vector embeddings using a transformer-based sentence embedding model.

The system uses a hybrid storage architecture to manage knowledge efficiently. Vector embeddings are stored in a vector database which allows semantic similarity search during query processing. At the same time, the relationships between concepts are maintained as structured triples within a graph-like representation of the knowledge base.

When a user submits a query, the system first converts the query into a vector embedding. This embedding is compared with stored embeddings to identify semantically similar knowledge capsules. The retrieved triples are then reconstructed into coherent responses.

The system also includes an audit component that evaluates the reliability of retrieved information. This module calculates a truth score which represents the confidence level of the generated response.

The modular architecture of the Living Library ensures that each component of the system performs a specific function. The main modules include text ingestion, triple extraction, semantic compression, embedding generation, hybrid storage, query processing, answer reconstruction, and audit verification. This architecture allows the system to efficiently manage knowledge while maintaining semantic relationships between concepts.

3.2 Architectural Design

Sequence Diagram – Query Processing and Response Generation

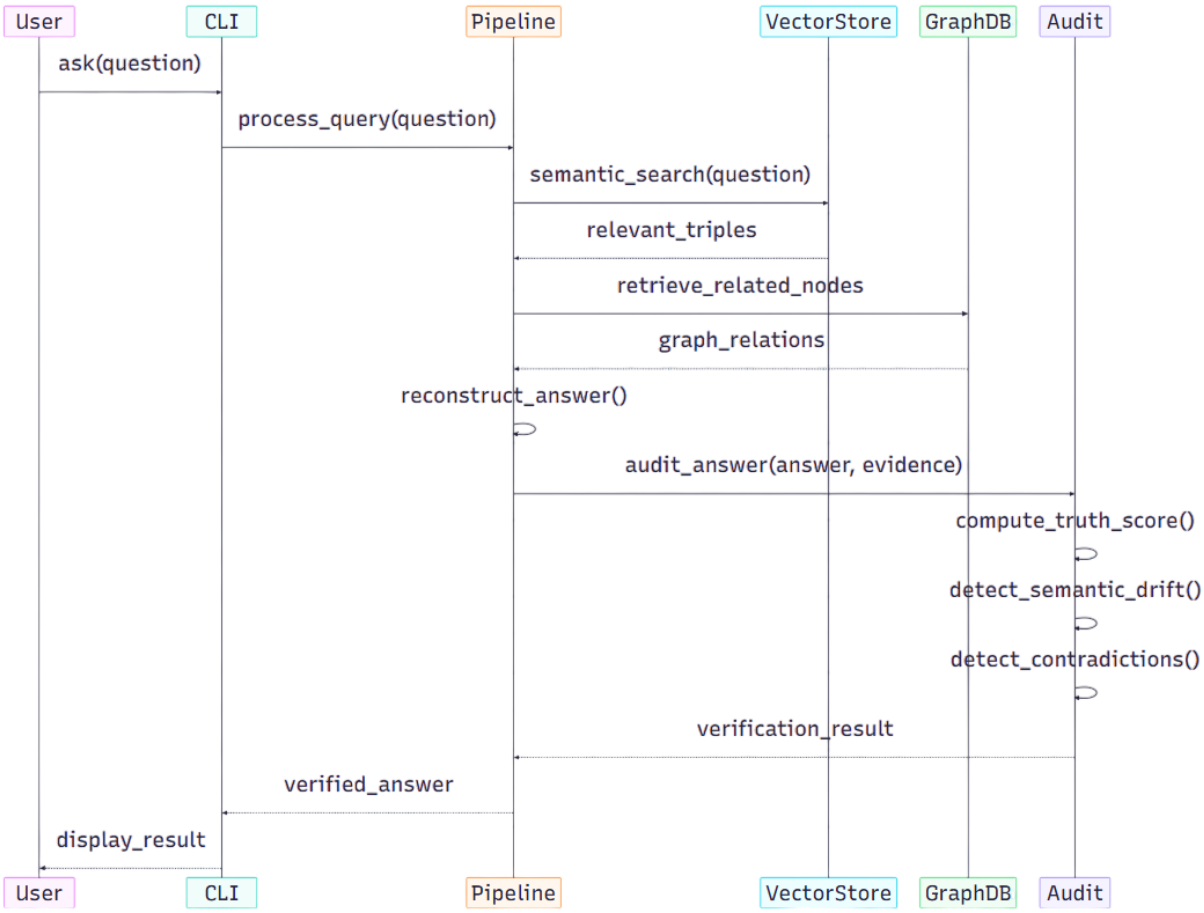


Figure 3.1: Sequence Diagram – Query Processing and Response Generation

System Architecture Diagram – Living Library Semantic Memory Pipeline

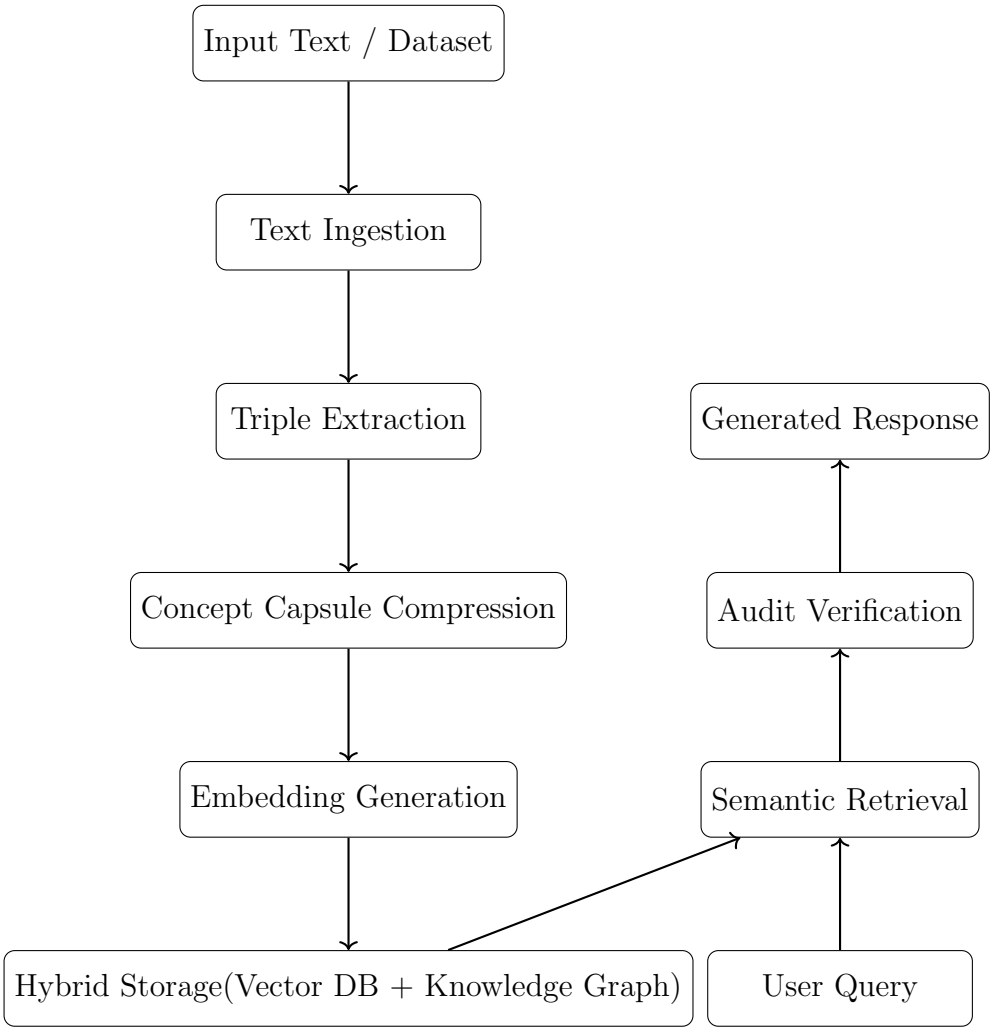


Figure 3.2: System Architecture of the Living Library Semantic Memory System

Data Flow Diagram – Knowledge Extraction and Storage Pipeline

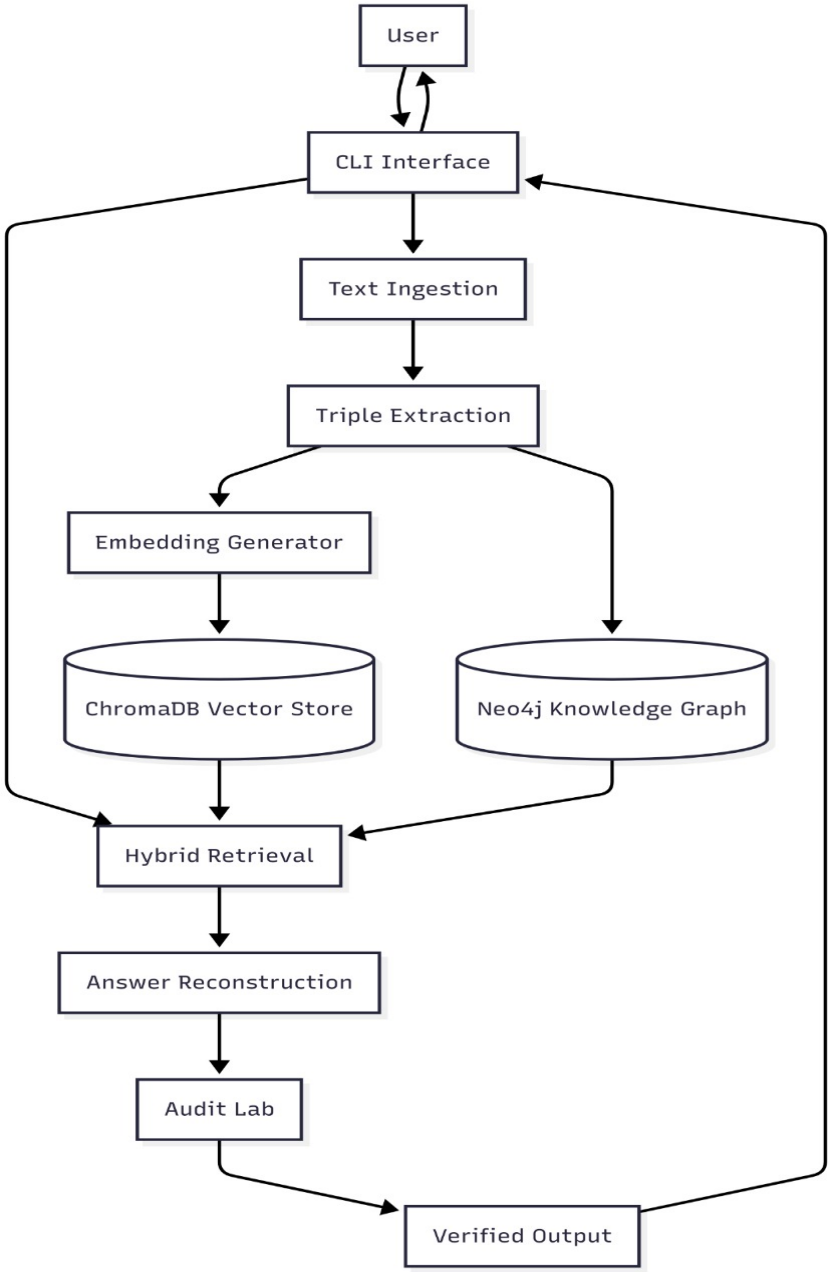


Figure 3.3: Data Flow Diagram of the Living Library System

3.3 Dataset identified

The Living Library system does not rely on a fixed or predefined dataset. Instead, it operates on user-provided textual input, which serves as the primary source of knowledge. For the prototype implementation, sample educational text documents were used to demonstrate the working of the system. These documents contain unstructured sen-

tences describing scientific and conceptual topics.

The input data is typically provided in plain text format and serves as the input to the system's ingestion pipeline. During processing, the text is segmented into individual sentences using natural language processing techniques. Each sentence is then analyzed to extract semantic relationships between entities.

From these sentences, subject–predicate–object triples are generated and stored as structured knowledge units. These triples represent relationships between concepts and form the basis of the system's knowledge base. The input text therefore acts as the source from which semantic knowledge is extracted and organized within the Living Library.

For example, a sentence such as *"Photosynthesis is the process by which plants convert light energy into chemical energy"* can be transformed into triples such as:

- (Photosynthesis, is, process)
- (Plants, convert, light energy)
- (Light energy, into, chemical energy)

These examples are used to demonstrate the system's ability to extract semantic relationships, compress knowledge, and retrieve relevant information during query processing.

3.4 Proposed Methodology/Algorithms

The Living Library system follows a structured multi-stage methodology to transform raw textual information into structured semantic knowledge.

3.4.1 Algorithm 1: Knowledge Extraction Process

Step 1: The system reads input documents and segments the text into sentences using natural language processing tools.(Text ingestion)

Step 2: Each sentence is analyzed to identify semantic relationships represented in the form of subject–predicate–object triples.(Triple Extraction)

Step 3: Extracted triples are grouped into concept capsules to reduce redundancy and improve storage efficiency.(Knowledge Compression)

Step 4: Each concept capsule is converted into a semantic vector representation using a sentence embedding model.(Embedding Generation)

3.4.2 Algorithm 2: Query Processing and Retrieval

Step 5: The generated embeddings are stored in a vector database for semantic search, while structured triples are preserved as relationships within the knowledge base.(Hybrid Storage)

Step 6: User queries are converted into vector embeddings and compared with stored embeddings to identify relevant knowledge capsules.(Query Processing)

Step 7: Retrieved triples are combined and reconstructed into meaningful responses.(Knowledge Reconstruction)

Step 8: The system evaluates retrieved knowledge and calculates a truth score indicating the reliability of the generated response.(Audit Verification)

3.5 User Interface Design

The user interface of the Living Library system is designed to provide a simple and intuitive interaction environment. The interface allows users to upload textual documents for knowledge ingestion as well as submit queries to retrieve stored information.

The interface contains an input panel where users can enter queries or upload text files. Once the input is provided, the system processes the information through the knowledge extraction pipeline and generates responses based on the stored knowledge base.

The output is displayed in a conversational format that allows users to easily understand the retrieved information. In addition to the query interface, the system includes a performance metrics panel that displays indicators such as truth score, accuracy measurements, and compression statistics.

The interface also supports visualization of knowledge relationships through a knowledge graph representation. This visualization allows users to observe connections between different concepts stored in the system.

3.6 Database Design

The Living Library system uses a hybrid database architecture that combines vector-based storage with structured knowledge representation. This approach enables efficient semantic search while preserving relationships between concepts.

The vector database stores embeddings generated from concept capsules. These embeddings represent semantic meaning and allow the system to perform similarity search when processing user queries. The vector database enables the system to retrieve relevant information even when the query does not exactly match stored text.

In addition to the vector database, the system maintains structured relationships between concepts in the form of subject–predicate–object triples. These relationships form a knowledge graph that preserves contextual connections between entities.

The hybrid storage architecture improves both retrieval performance and knowledge organization. By combining semantic embeddings with graph-based relationships, the system can support efficient information retrieval while maintaining structured knowledge representation.

3.7 Description of Implementation Strategies

The implementation of the Living Library system follows a modular pipeline strategy. Each stage of the system performs a specific function in transforming raw text into structured knowledge.

The system is implemented using Python due to its strong support for natural language processing and machine learning libraries. Sentence segmentation and linguistic processing are performed using the spaCy library. Semantic embeddings are generated using a transformer-based sentence embedding model.

The system stores embeddings in a vector database which supports semantic similarity search. Knowledge triples are managed through a graph-based representation which allows the system to preserve relationships between concepts.

The implementation also includes an audit module that verifies retrieved information and computes a truth score indicating the confidence level of the generated response. The modular design allows each component of the system to be developed and improved independently while maintaining the overall functionality of the Living Library framework.

3.8 Module Division

To simplify the development process, the Living Library system is divided into several functional modules. Each module performs a specific task in the overall pipeline, starting from reading the input text to verifying the correctness of the retrieved information. Dividing the system into modules helps in organizing the development work and allows each team member to focus on a particular component of the project.

3.8.1 Text Ingestion Module

The text ingestion module is responsible for reading the input data and preparing it for further processing. The system accepts textual information from an input file and processes it using natural language processing techniques. The spaCy library is used to break the input text into individual sentences so that each sentence can be analyzed separately in later stages.

3.8.2 Knowledge Extraction Module

This module focuses on extracting structured information from the input sentences. Each sentence is analyzed to identify important entities and relationships. The extracted information is then represented as subject–relation–object triples. These triples form the basic knowledge units that are stored in the system.

3.8.3 Embedding and Compression Module

Once the triples are extracted, they are converted into numerical vector representations called embeddings. These embeddings allow the system to understand semantic similarity between different pieces of information. Sentence-BERT is used to generate these embeddings, enabling efficient similarity comparison between stored knowledge and user queries. This step helps reduce redundancy in stored knowledge while preserving the essential meaning, thereby improving storage efficiency.

3.8.4 Knowledge Storage Module

The knowledge storage module manages how the extracted triples and their embeddings are stored. A hybrid storage approach is used in the system. Vector embeddings are

stored in ChromaDB to support semantic search, while the relationships between entities can be represented in a graph structure. This combination helps in efficient knowledge retrieval.

3.8.5 Query Processing and Retrieval Module

This module handles user queries. When a query is given, it is converted into an embedding and compared with the embeddings stored in the system. The most relevant triples are retrieved and combined to form a meaningful response for the user.

3.8.6 Audit and Verification Module

The audit module is responsible for checking the reliability of the generated answers. It compares the retrieved information with the stored knowledge base and calculates a truth score based on semantic similarity. This process helps identify unsupported or incorrect claims and improves the overall reliability of the system.

3.8.7 Module Allocation

The project work is divided among the team members so that each member contributes to the development of specific modules. This approach ensures balanced workload distribution and allows the system components to be developed and tested efficiently.

Project Member	**Primary Contributions**
Mohammed Yazid Zain	Data ingestion implementation, sentence segmentation, and initial triple extraction development.
Nikitha Praveen Basil	Semantic compression implementation and generation of Sentence-BERT embeddings for extracted triples.
Niya Elizabeth Tomy	Design of vector storage structure, semantic similarity retrieval, and improvements to the knowledge storage module.
Rebecca Oommen	Integration of system modules into a unified pipeline, development of query workflow, and implementation of the audit and verification module.

Table 3.1: Module Contribution Distribution

3.8.8 Work Schedule – Gantt Chart

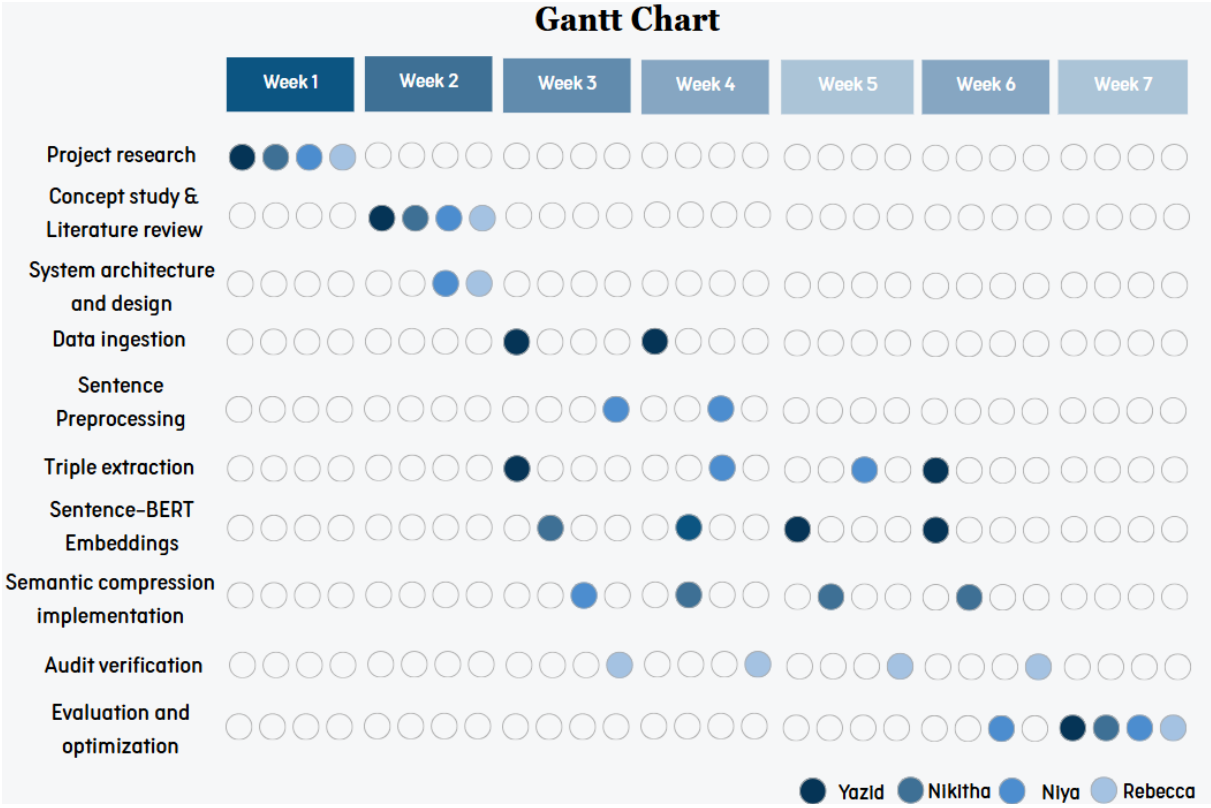


Figure 3.4: Project Work Schedule for the Living Library System

The development of the Living Library system was carried out over several stages, starting from initial research and system design to implementation, integration, and testing. A Gantt chart was prepared to plan the timeline of these activities and to monitor the progress of the project throughout the development period.

Chapter 4

Results and Discussions

4.1 Overview

The developed Living Library system successfully demonstrates the transformation of unstructured textual input into structured semantic knowledge. The system processes input text, extracts meaningful subject–predicate–object triples, and stores them in a structured format for efficient retrieval. Using embedding-based similarity search, the system is able to retrieve relevant information based on the meaning of user queries rather than exact keyword matching. The integration of an audit mechanism further enhances the reliability of the system by validating the generated responses against the stored knowledge. Overall, the results indicate that the system is capable of organizing knowledge effectively, reducing redundancy, and providing accurate and meaningful outputs during query processing.

4.2 Testing

The system was tested using sample textual inputs to evaluate different stages of the pipeline, including ingestion, triple extraction, semantic retrieval, and audit verification. Different types of queries were tested to observe the system's ability to retrieve relevant information based on semantic similarity. The performance metrics and visualizations further help in analyzing the effectiveness and reliability of the system. The following outputs demonstrate the functionality of the system.

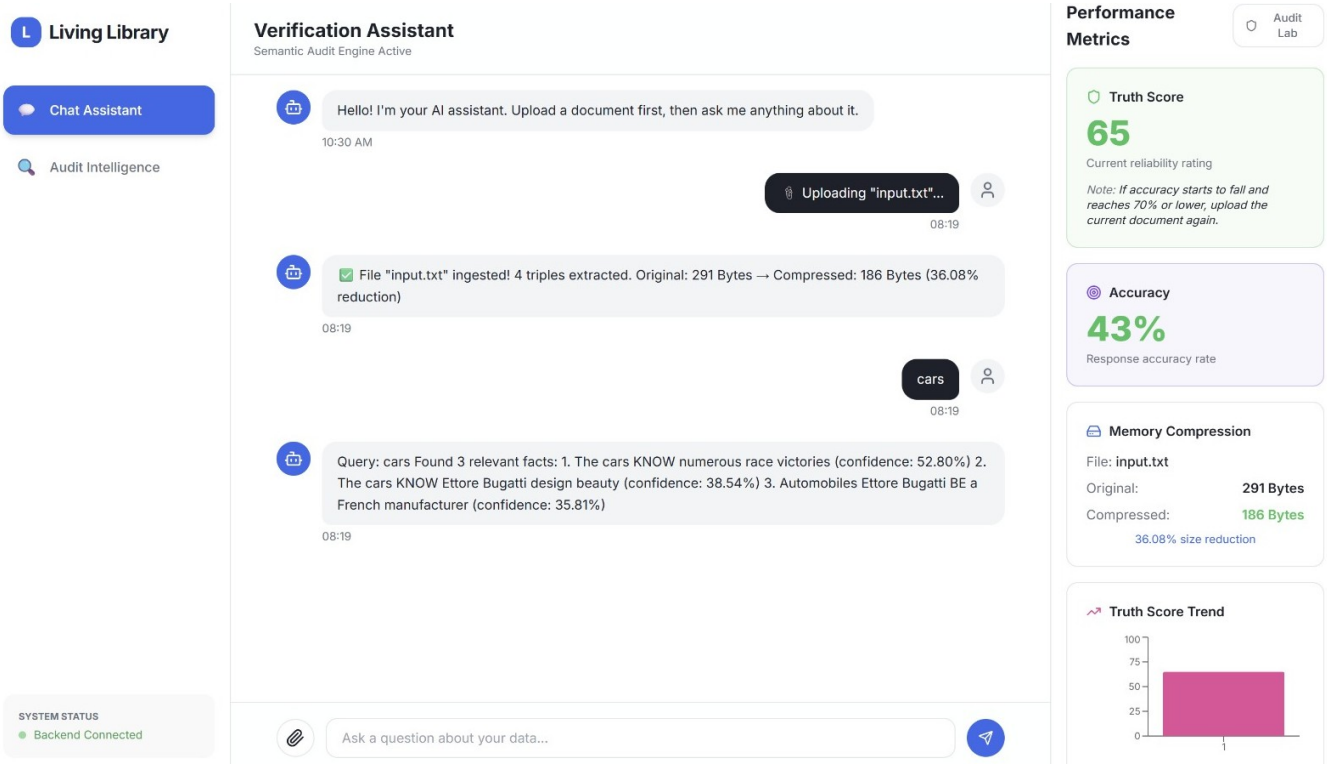


Figure 4.1: System response showing ingestion of input text, extraction of triples, query-based retrieval, and performance metrics such as compression and accuracy.

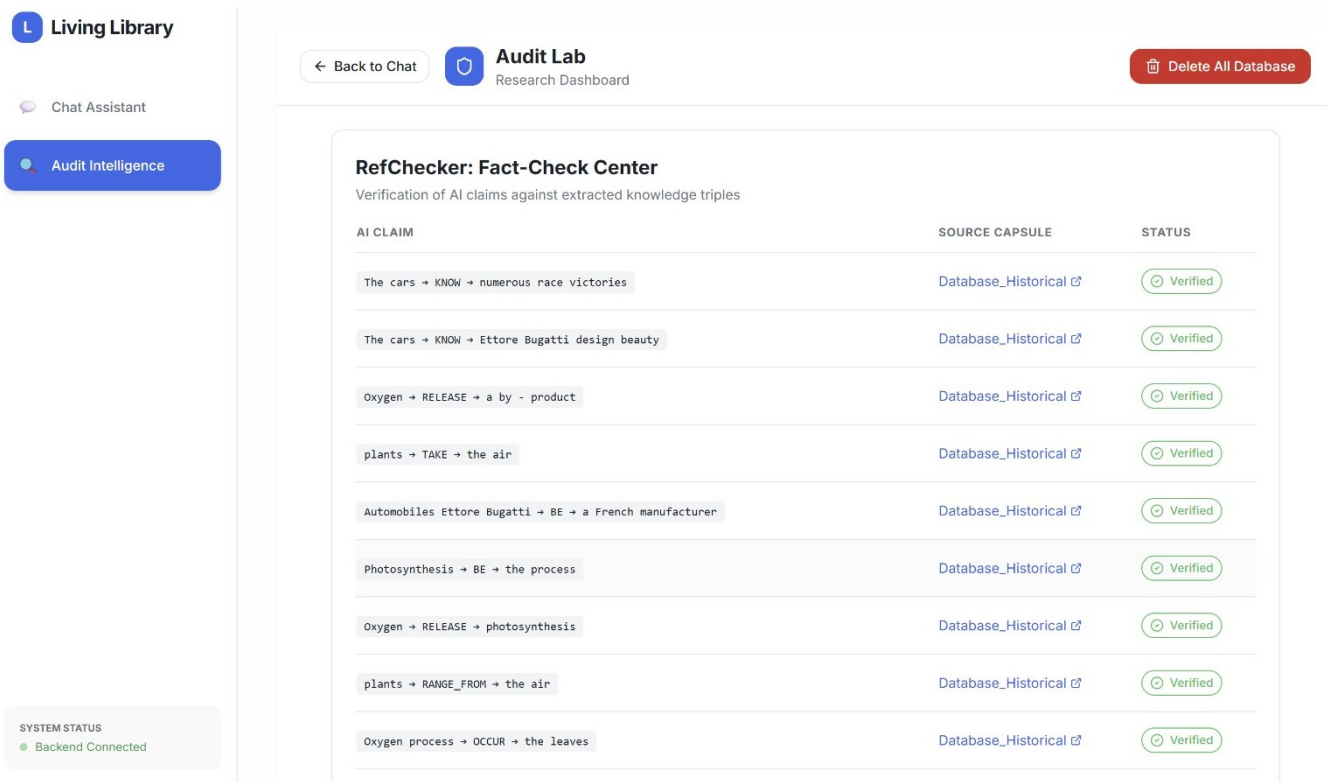


Figure 4.2: Audit Lab interface displaying fact-checking of extracted triples along with their verification status.

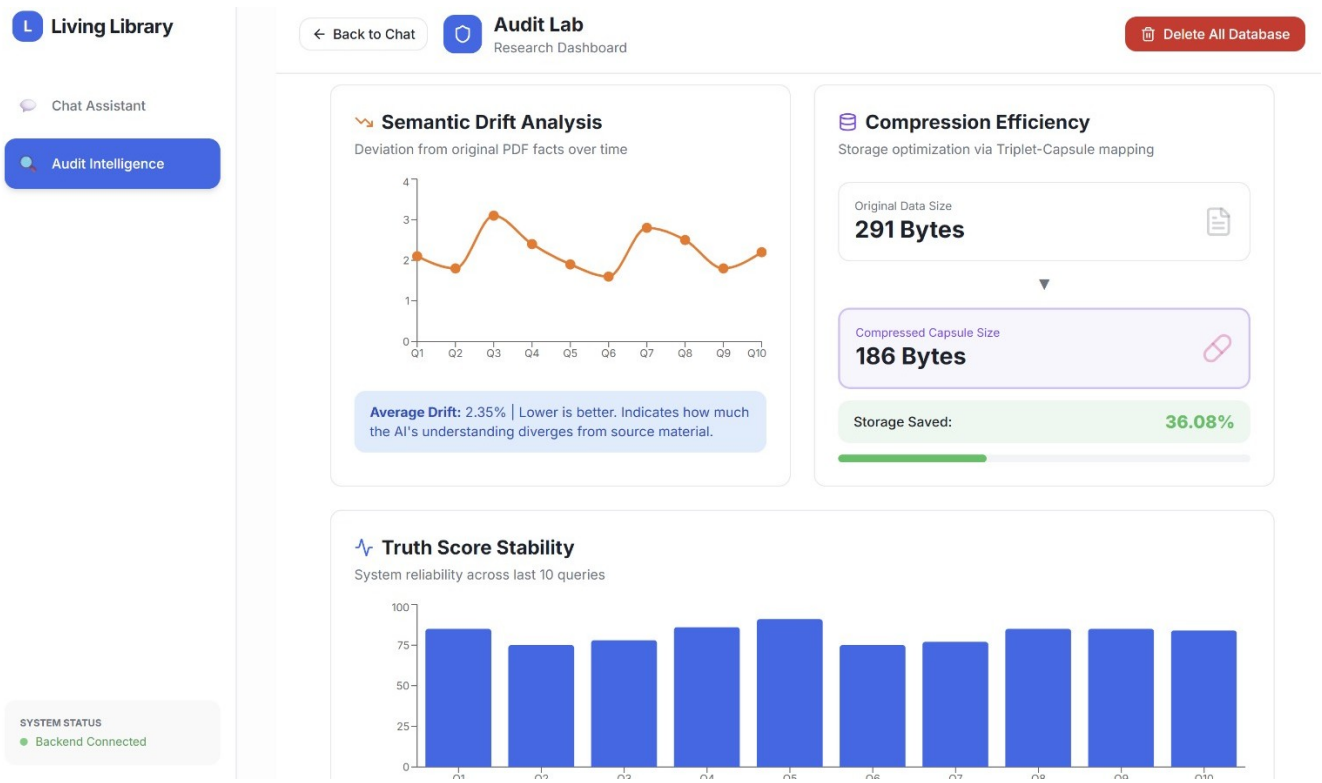


Figure 4.3: Semantic drift analysis, compression efficiency, and truth score stability visualizations showing system performance.

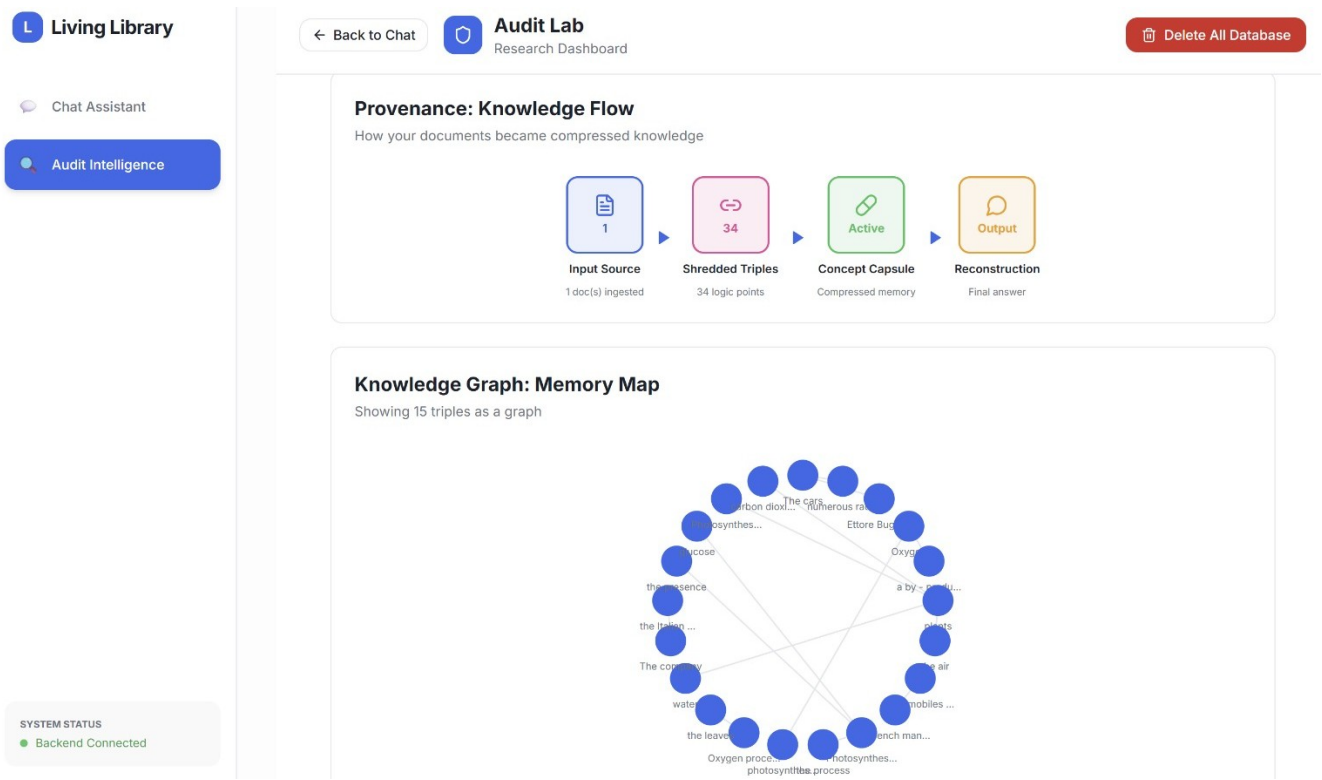


Figure 4.4: Knowledge graph and provenance flow representing extracted triples and their transformation into structured knowledge.

These results demonstrate that the system effectively extracts structured knowledge from unstructured text, performs semantic retrieval, and validates outputs using the audit mechanism. The visualizations further confirm the system's ability to manage knowledge efficiently and maintain reliability.

4.3 Quantitative Results

The performance of the Living Library system is evaluated using key quantitative metrics obtained during execution. These metrics provide insights into the system's accuracy, reliability, and compression efficiency.

Metric	Value
Truth Score	65
Accuracy	43%
Original Size	291 Bytes
Compressed Size	186 Bytes
Compression Rate	36.08%

Table 4.1: Quantitative performance metrics of the system

The truth score of 65 indicates that the system is able to validate generated responses with a moderate level of reliability. The accuracy value of 43% reflects the system's ability to retrieve relevant information based on semantic similarity rather than exact keyword matching. The compression results show that the system effectively reduces data size from 291 bytes to 186 bytes, achieving a compression rate of 36.08%. This demonstrates the efficiency of the concept capsule approach in minimizing redundancy while preserving meaningful information.

Based on the above findings, these quantitative results indicate that the system performs effectively in terms of knowledge representation, retrieval, and storage optimization.

4.4 Graphical Analysis

The performance of the Living Library system is analyzed using graphical representations derived from the obtained results. These visualizations provide a clearer understanding of the system's behavior in terms of accuracy, reliability, and compression efficiency. They also help in comparing different aspects of the system in a more intuitive and interpretable manner.

The performance metrics graph presents the truth score and accuracy achieved by the system. The truth score of 65 indicates that the audit mechanism is able to validate the generated responses to a reasonable extent. This shows that most of the retrieved information is consistent with the stored knowledge. The accuracy value of 43% reflects the system's ability to retrieve relevant information based on semantic similarity, even when exact keyword matches are not present. This highlights the effectiveness of embedding-based retrieval in understanding the context of queries.

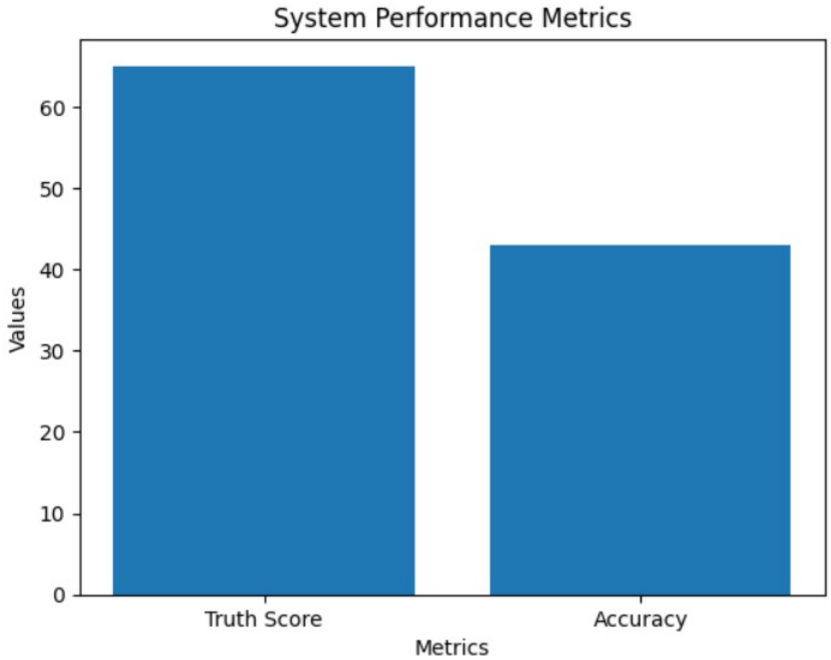


Figure 4.5: Performance metrics showing truth score and accuracy of the system.

The compression comparison graph highlights the reduction in data size achieved by the system. The original input size of 291 bytes is reduced to 186 bytes after processing. This demonstrates the effectiveness of the concept capsule approach in eliminating redundancy while preserving essential information.

This reduction improves storage and retrieval efficiency, allowing the system to process queries faster. By retaining only meaningful relationships, the system maintains the quality of results while minimizing storage requirements.

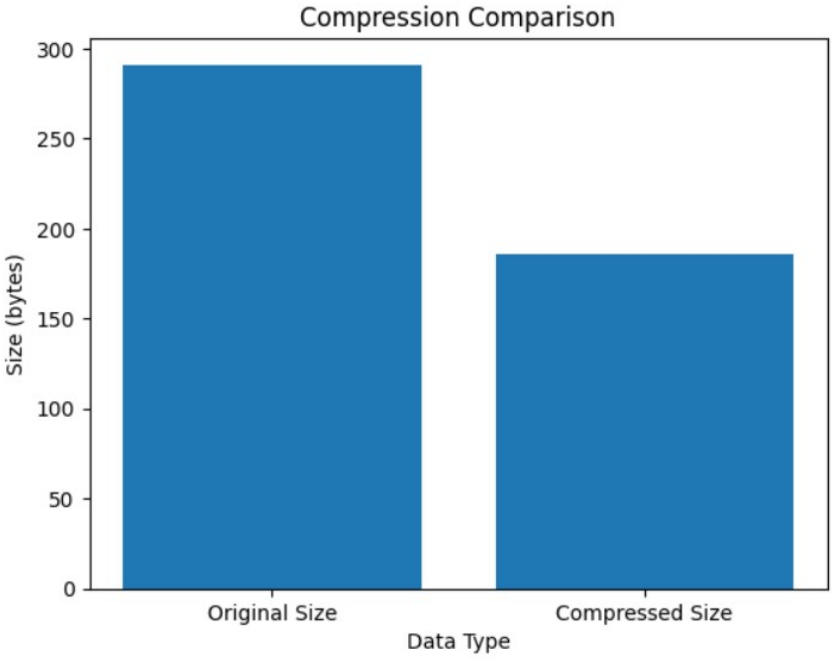


Figure 4.6: Comparison of original and compressed data sizes.

The compression distribution graph provides a visual representation of the proportion of data reduced and retained after processing. The system achieves a compression rate of 36.08%, indicating a significant reduction in storage requirements while maintaining the essential semantic content.

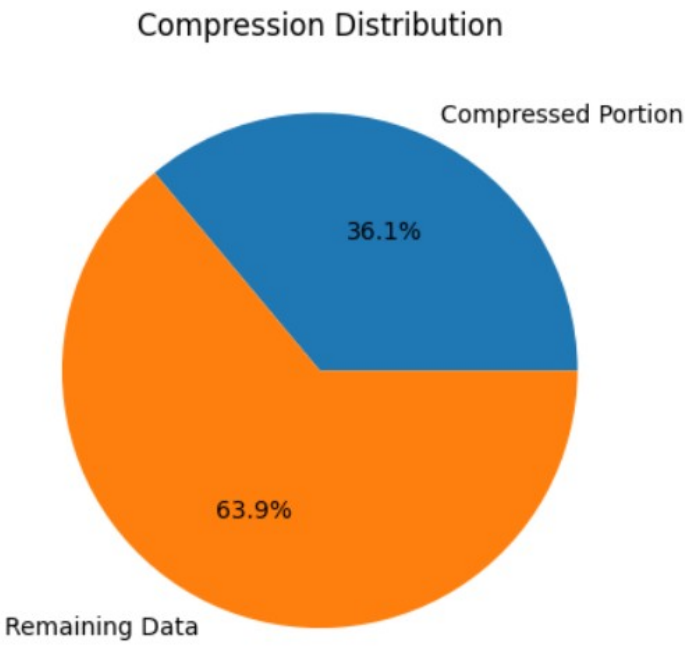


Figure 4.7: Compression distribution showing percentage of data reduced and retained after processing.

Thus, the graphical analysis confirms that the system performs efficiently across multiple aspects, including knowledge compression, semantic retrieval, and response validation. The results indicate that the system is able to balance storage efficiency with retrieval accuracy, making it effective for semantic knowledge management.

4.5 Discussion

The results obtained demonstrate that the system is effective in transforming unstructured text into structured semantic knowledge. The extraction of subject–predicate–object triples allows the system to represent information in a meaningful and compact form. The use of embedding-based retrieval enables the system to return relevant results even when queries are phrased differently, indicating good semantic understanding.

The audit mechanism plays an important role in improving the reliability of the system by verifying generated outputs against stored knowledge. The performance metrics, such as truth score and accuracy, provide useful insights into the system's behavior and help in evaluating its consistency. The compression results also show that the system is able to reduce storage size while maintaining essential information.

However, the system may face limitations when processing highly complex or ambiguous sentences, which can affect the accuracy of triple extraction. Despite this, the overall performance indicates that the system provides a strong foundation for semantic knowledge storage, retrieval, and verification, with scope for further improvement.

Chapter 5

Conclusion

5.1 Conclusion

The Living Library project focuses on converting unstructured text into a more meaningful and organized form of knowledge. Instead of storing complete documents, the system extracts important relationships from the text in the form of subject–predicate–object triples. These triples represent the core meaning of the content and allow the system to break down large amounts of text into smaller, manageable units of knowledge. The extracted triples are further organized into compact representations, which helps in reducing redundancy while still preserving the essential meaning. This approach makes the system more structured compared to traditional text-based systems, where information is often stored without any clear organization.

To improve retrieval, the system uses embedding techniques to convert the structured knowledge into vector form. This allows the system to perform semantic search, where results are based on meaning rather than exact word matching. The use of both vector-based storage and graph-based representation provides a balance between flexibility and structure, making the system effective in handling different types of queries. Another important feature of the project is the audit mechanism. This component checks the generated responses against the stored knowledge and helps identify inconsistencies or missing information. It plays a key role in improving the reliability of the system by reducing incorrect outputs.

Overall, the Living Library project demonstrates how semantic techniques can be used to improve the way knowledge is stored and retrieved. It highlights the importance of structuring information and ensuring reliability in AI-based systems.

5.2 Future Scope

The system can be further improved by extending its capability to handle different types of data such as images, audio, and structured datasets, in addition to text. The accuracy of triple extraction can be enhanced by using more advanced natural language processing techniques and better parsing methods. The audit mechanism can also be improved by incorporating real-time validation using external knowledge sources. In addition, the system can be scaled to support larger datasets and faster query processing. Future work can also focus on improving the visualization of knowledge graphs and developing a user-friendly interface, making the system more practical for real-world applications.

References

[1] Y. Hao, Z. Li and Y. Zhang, "MELODI: Exploring Memory Compression for Long Contexts," arXiv preprint arXiv:2410.03156, Apr. 2025. Available: https://arxiv.org/pdf/2410.03156.

[2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," arXiv preprint arXiv:1908.10084, Aug. 2019. Available: https://arxiv.org/pdf/1908.10084.

[3] Z. Yang et al., "LongMemEval: Benchmarking Long-Term Interactive Memory in Large Language Models," in Proc. International Conference on Learning Representations (ICLR), 2025.

[4] X. Chen et al., "Think-on-Graph 3.0: Adaptive Graph Reasoning for Large Language Models," arXiv preprint arXiv:2509.21710, Sept. 2025. Available: https://arxiv.org/pdf/2509.21710.

Appendix A: Presentation

Living Library: A Semantic Memory System for Efficient Knowledge Storage

FINAL PRESENTATION

Project Guide:
Ms. Janani K

Team members:
Mohammed Yazid Zain
Nikitha Praveen Basil
Niya Elizabeth Tomy
Rebecca Oommen

Contents

01 Problem Definition
02 Problem Objective
03 Purpose and Need
04 Literature Survey
05 Proposed Method
06 Architecture Diagram
07 Design Diagram
08 Result and Output
09 Assumptions
10 Work Breakdown

Contents

11 Requirements

12 Gantt Chart

13 Risks and Challenges

14 Conclusion

15 References

Introduction

THE PROBLEM:

- **Modern AI systems**
  - **struggle to handle large documents efficiently (memory limit).**
  - **store and search raw text - slow and costly.**

OUR SOLUTION:

"Living Library"

- **stores only core semantic facts instead of full documents.**
- **This reduces storage and allows AI to reconstruct explanations when needed.**
- **Domain of Interest: AI Memory and NLP.**

Problem Definition

To develop a semantic memory framework that stores knowledge as compressed concept capsules instead of raw text for efficient long-term AI memory - validated by an audit loop for accuracy checks.

Objectives

- Identify limitations of raw text storage in long-term AI memory.
- Design a semantic memory framework using concept capsules.
- Implement efficient storage and retrieval of concept capsules.
- Develop an audit loop for memory validation and accuracy checking.
- Evaluate framework performance against traditional text-based memory systems.

Purpose and Need

Purpose:
To develop a semantic knowledge system that converts unstructured text into structured concept capsules and provides verified, explainable answers using hybrid vector and graph retrieval.

Need:

- Similarity-based retrieval can return unverified information.
- Large text storage causes redundancy and inefficiency.
- AI answers need verification and explainable evidence.
- Hybrid retrieval (vector + graph) enables more reliable knowledge access.

Literature Survey

MELODI: Exploring Memory Compression for Long Contexts

Core Idea

- Uses memory compression to convert long text into compact semantic representations.
- Stores key information while preserving important relationships.

Contribution

- Reduces storage and computational cost for long-context reasoning.
- Shows that compressed memory can still support effective retrieval.

Limitations

- Compression may remove fine-grained details from the original text.
- Performance depends on how well the compression preserves important information.

Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks

Core Idea

- Uses a Siamese BERT architecture to generate sentence embeddings.
- Allows efficient sentence similarity comparison using vector distance.

Contribution

- Introduces Sentence-BERT for generating meaningful sentence embeddings.
- Greatly improves speed for semantic similarity and clustering tasks.

Limitations

- Embedding quality depends on the training data and fine-tuning.
- May not capture very complex contextual relationships between sentences.

LongMemEval: Benchmarking Long-Term Interactive Memory in Large Language Models

Core Idea

- Introduces a benchmark to evaluate long-term memory in LLMs.
- Tests whether models can recall information across long conversations.

Contribution

- Provides a standardized benchmark for long-term interactive memory evaluation.
- Reveals limitations of current LLMs in retaining earlier context.

Limitations

- Focuses only on evaluation, not improving memory methods.
- Benchmark tasks may not fully reflect real-world scenarios.

Think-on-Graph 3.0: Adaptive Graph Reasoning for Large Language Models

Core Idea

- Represents knowledge as a graph of nodes and relationships.
- Uses adaptive traversal to guide LLM reasoning.

Contribution

- Improves multi-hop reasoning using graph structure.
- Makes reasoning paths more interpretable.

Limitations

- Depends on the quality of the knowledge graph.
- Adds extra computational overhead.

Proposed Method

Step 1: Text Ingestion and Preprocessing

The system first ingests raw textual documents provided by the user. The input text is processed by splitting it into individual sentences using a sentence segmentation module. This preprocessing step prepares the document for structured knowledge extraction.

Step 2: Semantic Triple Extraction

Each sentence is converted into a Subject–Relation–Object (SRO) triple using a rule-based extraction approach. These triples represent the core factual relationships contained in the text. The extracted triples act as semantic concept capsules, ensuring that only meaningful factual information is retained instead of storing the entire raw text.

Step 3: Semantic Compression and Embedding Generation

The extracted triples serve as a compressed representation of the original text. Each triple is converted into a vector embedding using Sentence-BERT (all-MiniLM-L6-v2), allowing semantic similarity comparison between queries and stored knowledge.

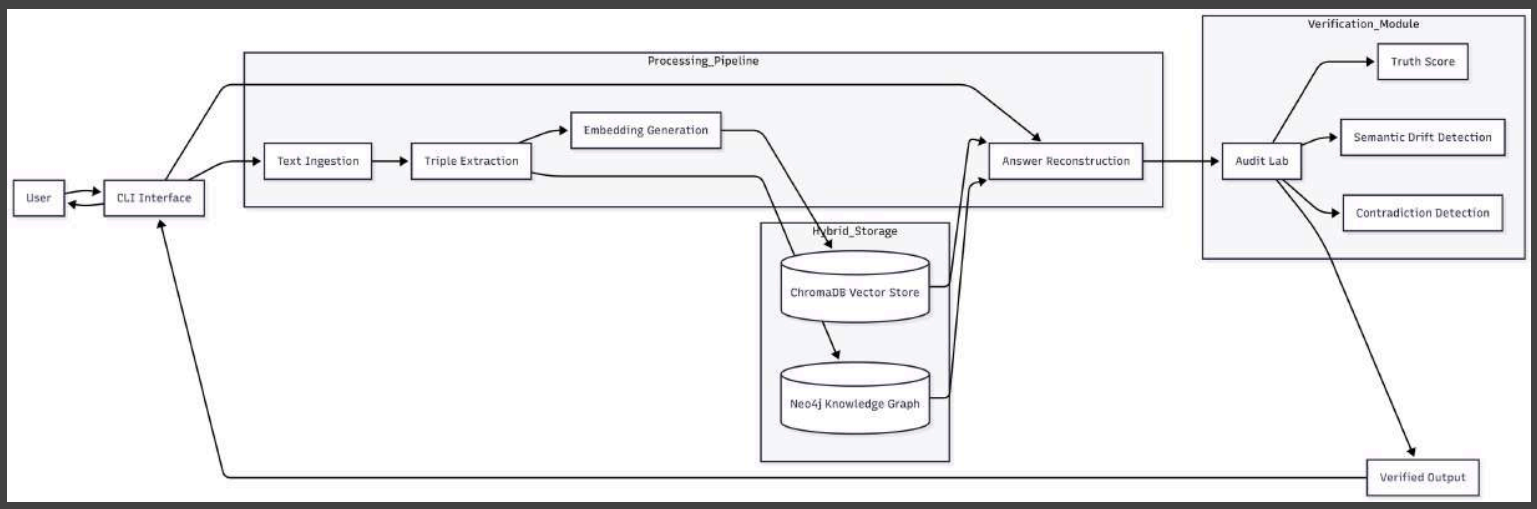
Step 4: Hybrid Knowledge Storage and Retrieval

The embeddings are stored in ChromaDB for semantic search, while the triples are also stored in Neo4j as a knowledge graph. When a query is submitted, the system retrieves the most relevant triples using semantic similarity and graph relationships.

Step 5: Answer Reconstruction

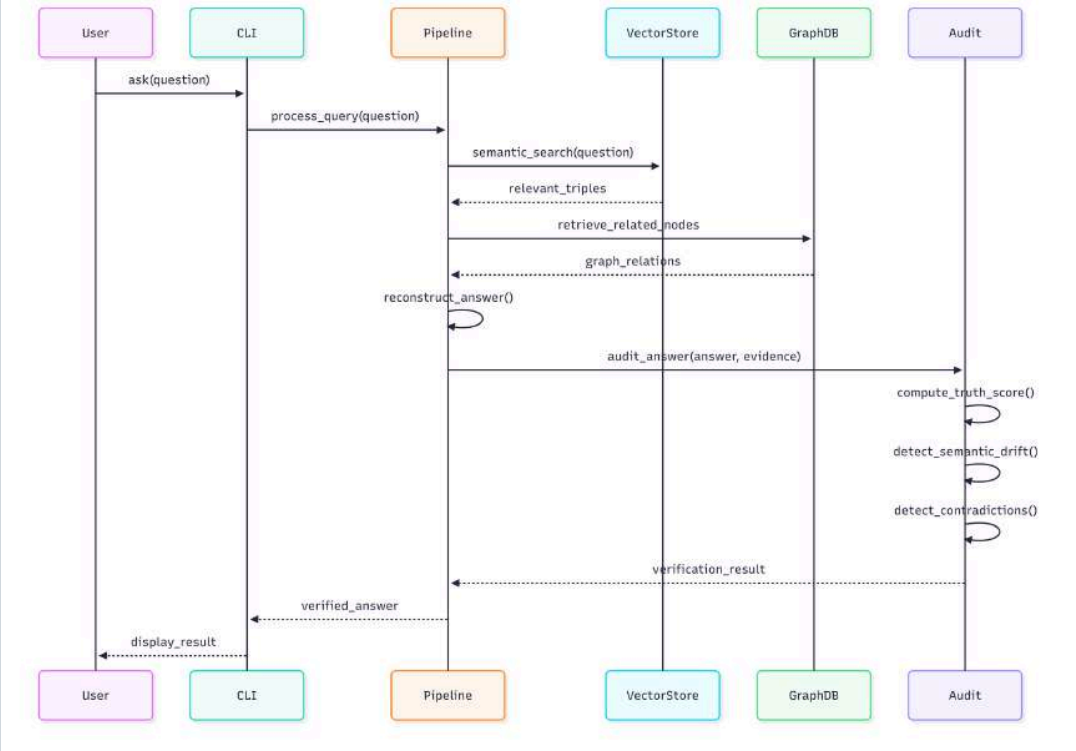
The system uses the retrieved triples from the hybrid memory to construct a meaningful response to the user's query. The relevant concept capsules are combined and organized into a coherent answer. The reconstructed answer is then passed to the audit module for verification.

Architecture Diagram

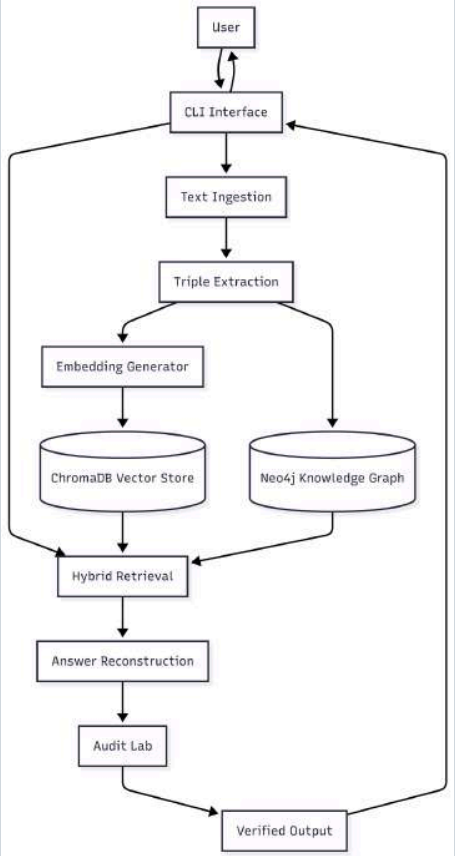


Design Diagram

Sequence Diagram



Data Flow Diagram



Result & Output

Living Library

Chat Assistant

Audit Intelligence

SYSTEM STATUS
Backend Connected

Verification Assistant

Semantic Audit Engine Active

Hello! I'm your AI assistant. Upload a document first, then ask me anything about it.

10:30 AM

Uploading "input.txt"...

08:19

File "input.txt" ingested! 4 triples extracted. Original: 291 Bytes → Compressed: 186 Bytes (36.08% reduction)

08:19

cars

08:19

Query: cars Found 3 relevant facts: 1. The cars KNOW numerous race victories (confidence: 52.80%) 2. The cars KNOW Ettore Bugatti design beauty (confidence: 38.54%) 3. Automobiles Ettore Bugatti BE a French manufacturer (confidence: 35.81%)

08:19

Ask a question about your data...

Performance Metrics

Audit Lab

Truth Score

65

Current reliability rating

Note: If accuracy starts to fall and reaches 70% or lower, upload the current document again.

Accuracy

43%

Response accuracy rate

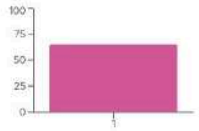
Memory Compression

File: input.txt

Original:	291 Bytes
Compressed:	186 Bytes

36.08% size reduction

Truth Score Trend



Living Library

Chat Assistant

Audit Intelligence

SYSTEM STATUS
Backend Connected

← Back to Chat

Audit Lab

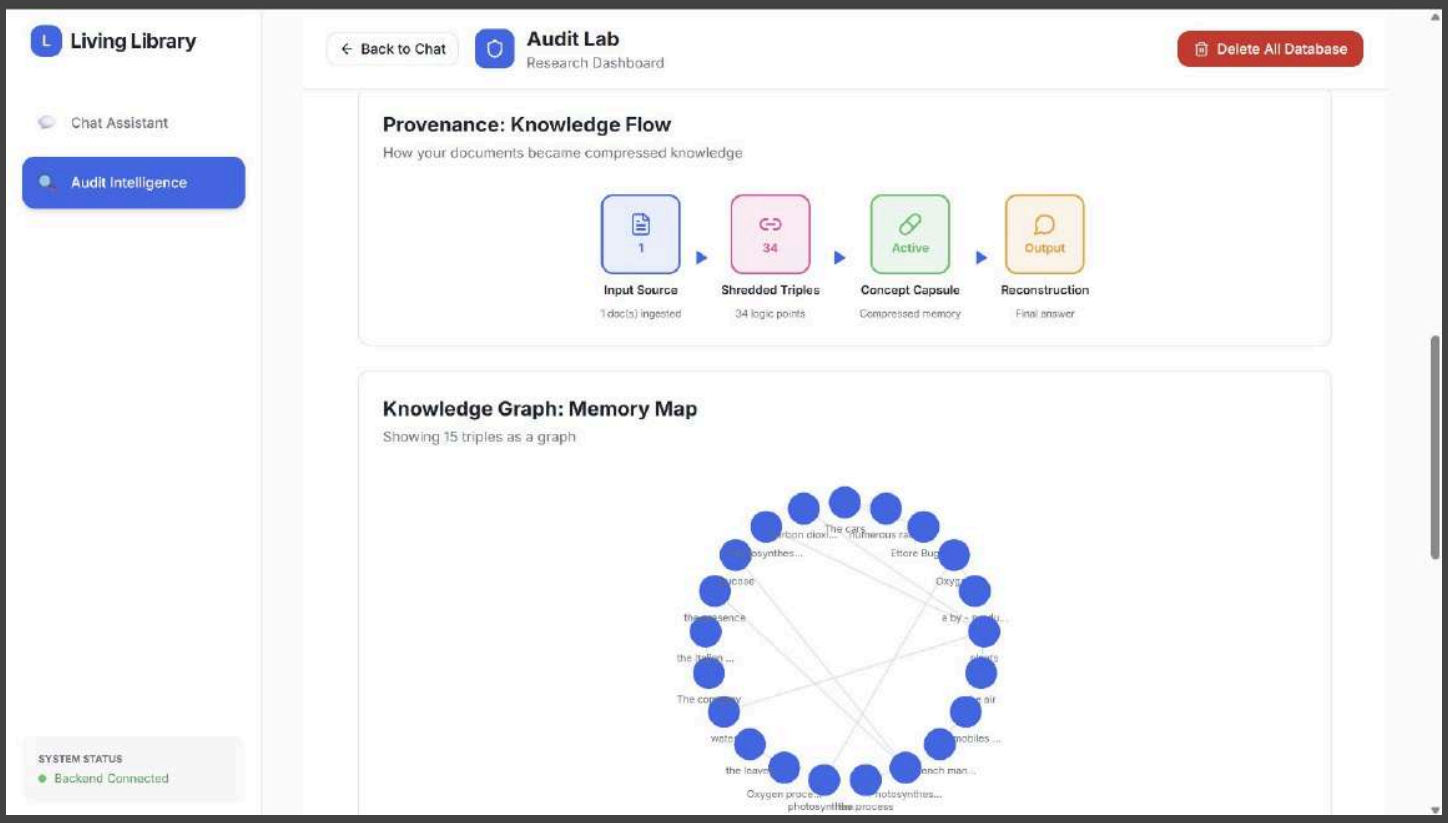
Research Dashboard

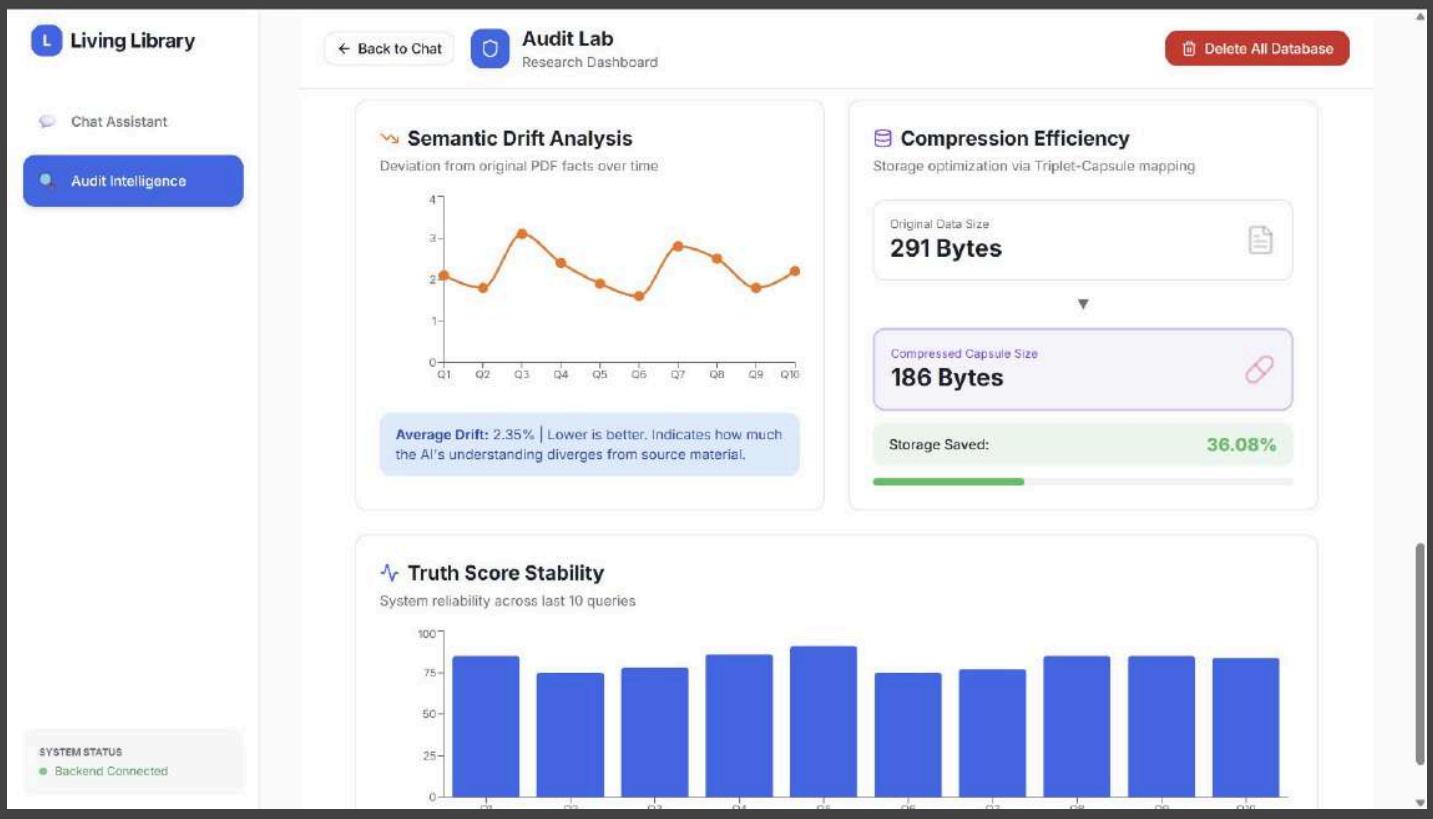
Delete All Database

RefChecker: Fact-Check Center

Verification of AI claims against extracted knowledge triples

AI CLAIM	SOURCE CAPSULE	STATUS
The cars → KNOW → numerous race victories	Database_Historical	Verified
The cars → KNOW → Ettore Bugatti design beauty	Database_Historical	Verified
Oxygen → RELEASE → a by - product	Database_Historical	Verified
plants → TAKE → the air	Database_Historical	Verified
Automobiles Ettore Bugatti → BE → a French manufacturer	Database_Historical	Verified
Photosynthesis → BE → the process	Database_Historical	Verified
Oxygen → RELEASE → photosynthesis	Database_Historical	Verified
plants → RANGE_FROM → the air	Database_Historical	Verified
Oxygen process → OCCUR → the leaves	Database_Historical	Verified


Living Library
Chat Assistant
Audit Intelligence
SYSTEM STATUS
Backend Connected
Back to Chat
Audit Lab
Research Dashboard
Delete All Database
Provenance: Knowledge Flow
How your documents became compressed knowledge
1
Input Source
1 doc(s) ingested
34
Shredded Triples
34 logic points
Active
Concept Capsule
Compressed memory
Output
Reconstruction
Final answer
Knowledge Graph: Memory Map
Showing 15 triples as a graph


Living Library
Chat Assistant
Audit Intelligence
SYSTEM STATUS
Backend Connected
Back to Chat
Audit Lab
Research Dashboard
Delete All Database
Semantic Drift Analysis
Deviation from original PDF facts over time
4
3
2
1
0
Q1
Q2
Q3
Q4
Q5
Q6
Q7
Q8
Q9
Q10
Average Drift: 2.35% | Lower is better. Indicates how much the AI's understanding diverges from source material.
Compression Efficiency
Storage optimization via Triplet-Capsule mapping
Original Data Size
291 Bytes
Compressed Capsule Size
186 Bytes
Storage Saved:
36.08%
Truth Score Stability
System reliability across last 10 queries
100
75
50
25
0

Assumptions

- Input documents contain accurate and meaningful information.
- Text can be converted into valid subject–relation–object triples.
- Semantic embeddings can capture similarity between queries and stored knowledge.
- The knowledge base is limited and domain-specific, not the entire web.
- Users ask questions related to the ingested documents.

Work Breakdown

Phase 1: Concept & Architecture

(December-January)

All members

- Research on semantic memory systems and knowledge retrieval.
- Study of semantic compression and vector embeddings.
- Understanding hybrid memory architecture.
- System architecture design for the Living Library framework.
- Identification of core modules: ingestion, triple extraction, embeddings, storage, and audit verification.

Phase 2: Core System Development

(January-February)

Yazid

- Data ingestion implementation.
- Sentence segmentation from input documents.
- Basic Subject–Relation–Object triple extraction.

Nikitha

- Semantic compression concept implementation.
- Sentence-BERT embedding generation for triples.
- Testing embedding outputs for semantic representation.

Phase 2: Core System Development

(January-February)

Niya

- Vector storage structure design.
- Semantic similarity retrieval implementation.
- Query encoding for similarity search.

Rebecca

- Integration of modules into a unified pipeline.
- Query processing workflow implementation.
- System execution testing for end-to-end flow.

Phase 3: Hybrid Memory & Verification Development

(February-March)

Yazid

- Embedding pipeline refinement using Sentence-BERT.
- Optimization of semantic compression and vector generation.
- Integration of embeddings with retrieval pipeline.

Nikitha

- Knowledge storage and retrieval module improvements.
- Integration with vector database for semantic search.
- Retrieval performance testing and validation.

Phase 3: Hybrid Memory & Verification Development

(February-March)

Niya

- Improvements to triple extraction module.
- Refinement of semantic fact generation.
- Frontend–backend integration using FastAPI.

Rebecca

- Development of Audit Lab verification module.
- Implementation of truth score, contradiction detection, and semantic drift analysis.
- System testing.

Phase 4: Evaluation & Optimization

(March)

All members

- Resolved Neo4j crash by switching to a local offline Neo4j instance.
- Improved triple extraction to support more sentence formats.
- Correct implementation of semantic memory compression.
- Added semantic drift analysis for response validation.
- Fixed FastAPI backend issues for stable system integration.
- Added confidence score and concept references for generated responses.
- Ongoing work on permanent knowledge storage.

Requirements

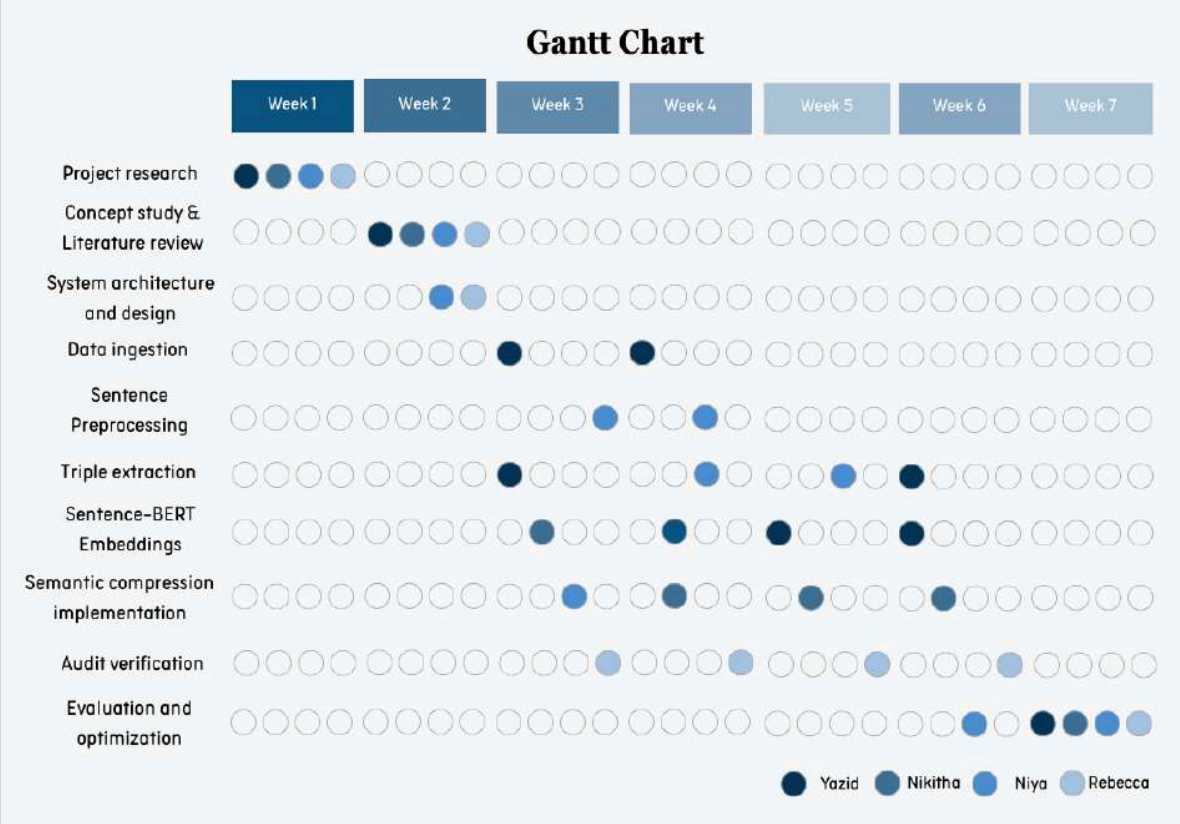
Software Requirements

- Python 3.10 or 3.11 or 3.12 (used 3.12)
- spaCy (for text processing and fact extraction)
- Sentence-BERT (for semantic similarity)
- BAAI/BGE-M3 (for compression)
- DeepSeek R1 Distill Llama 8B (Shredder)
- DeepSeek R1 Distill 70B (validation)
- ChromaDB (for vector storage)
- Jupyter Notebook / VS Code

Hardware Requirements

- Standard PC or Laptop
- Minimum 8 GB RAM
- Intel i5 / equivalent processor
- 10 GB free disk space

Gantt Chart


Gantt Chart
Week 1
Week 2
Week 3
Week 4
Week 5
Week 6
Week 7
Project research
Concept study & Literature review
System architecture and design
Data ingestion
Sentence Preprocessing
Triple extraction
Sentence-BERT Embeddings
Semantic compression implementation
Audit verification
Evaluation and optimization
Yazid
Nikitha
Niya
Rebecca

Risks and Challenges

Risks and Challenges

- **Neo4j Cloud Downtime:** Neo4j Aura instance became unavailable for several days, interrupting graph storage and testing.
- **Component Integration Issues:** Integrating FastAPI, ChromaDB, Neo4j, and the frontend required extensive debugging.
- **File Ingestion and Persistence Issues:** Some uploaded files failed to process correctly and were not stored permanently, requiring repeated uploads.

Risks and Challenges

- **Extraction Rule Limitations:** The rule-based triple extraction could not process certain sentence structures or long, complex sentences.
- **Version Compatibility Conflicts:** Dependency conflicts between Python libraries required environment adjustments.
- **Frontend Visualization Limitations:** The interface initially did not clearly display compressed memory and extracted knowledge.

Conclusion

The Living Library system converts unstructured text into structured semantic triples that capture meaningful factual relationships. These triples are embedded using Sentence-BERT and stored in a hybrid architecture combining ChromaDB for semantic search and Neo4j for relationship modeling.

The system retrieves relevant knowledge through semantic similarity and reconstructs answers from stored triples. An audit verification module then evaluates the response using truth scoring, contradiction detection, and evidence tracing to ensure reliable and explainable results.

References

[1] Y. Hao, Z. Li, and Y. Zhang, “MELODI: Exploring Memory Compression for Long Contexts,” *arXiv preprint arXiv:2410.03156*, Apr. 2025. Available: https://arxiv.org/pdf/2410.03156
[2] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” *arXiv preprint arXiv:1908.10084*, Aug. 2019. Available: https://arxiv.org/pdf/1908.10084
[3] Z. Yang et al., “LongMemEval: Benchmarking Long-Term Interactive Memory in Large Language Models,” in *Proc. Int. Conf. Learning Representations* (ICLR), 2025.
[4] X. Chen et al., “Think-on-Graph 3.0: Adaptive Graph Reasoning for Large Language Models,” *arXiv preprint arXiv:2509.21710,* Sept. 2025. Available: https://arxiv.org/pdf/2509.21710

Thank You

for your time
and attention

Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
RAJAGIRI SCHOOL OF ENGINEERING & TECHNOLOGY (AUTONOMOUS)
RAJAGIRI VALLEY, KAKKANAD, KOCHI, 682039
(Affiliated to APJ Abdul Kalam Technological University)

Vision, Mission, Programme Outcomes and Course Outcomes

Institute Vision

To evolve into a premier technological institution, moulding eminent professionals with creative minds, innovative ideas and sound practical skill, and to shape a future where technology works for the enrichment of mankind.

Institute Mission

To impart state-of-the-art knowledge to individuals in various technological disciplines and to inculcate in them a high degree of social consciousness and human values, thereby enabling them to face the challenges of life with courage and conviction.

Department Vision

To become a centre of excellence in Computer Science and Engineering, moulding professionals catering to the research and professional needs of national and international organizations.

Department Mission

To inspire and nurture students, with up-to-date knowledge in Computer Science and

Engineering, ethics, team spirit, leadership abilities, innovation and creativity to come out with solutions meeting societal needs.

Programme Outcomes (PO)
Engineering Graduates will be able to:

1. Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4. Conduct investigations of complex problems: Use research-based knowledge including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. Modern Tool Usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal, and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. Individual and Team work: Function effectively as an individual, and as a member or leader in teams, and in multidisciplinary settings.

10. Communication: Communicate effectively with the engineering community and with society at large. Be able to comprehend and write effective reports documentation. Make effective presentations, and give and receive clear instructions.

11. Project management and finance: Demonstrate knowledge and understanding of engineering and management principles and apply these to one's own work, as a member and leader in a team. Manage projects in multidisciplinary environments.

12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and lifelong learning in the broadest context of technological change.

Programme Specific Outcomes (PSO)

A graduate of the Computer Science and Engineering Program will demonstrate:

PSO1: Computer Science Specific Skills

The ability to identify, analyze and design solutions for complex engineering problems in multidisciplinary areas by understanding the core principles and concepts of computer science and thereby engage in national grand challenges.

PSO2: Programming and Software Development Skills

The ability to acquire programming efficiency by designing algorithms and applying standard practices in software project development to deliver quality software products meeting the demands of the industry.

PSO3: Professional Skills

The ability to apply the fundamentals of computer science in competitive research and to develop innovative products to meet the societal needs thereby evolving as an eminent researcher and entrepreneur.

Course Outcomes

CO1: Identify technically and economically feasible problems.

CO2: Identify and survey the relevant literature for getting exposed to related solutions and software development processes.

CO3: Perform requirement analysis, identify design methodologies and develop adaptable and reusable solutions of minimal complexity by using modern tools and advanced programming techniques.

CO4: Prepare technical reports and deliver presentations.

CO5: Apply engineering and management principles to achieve the goal of the project.

Appendix C: CO-PO-PSO Mapping

COURSE OUTCOMES:

Sl.No	DESCRIPTION	Blooms' Taxonomy Level
CO1	Identify technically and economically feasible problems.	Apply
CO2	Identify and survey the relevant literature for getting exposed to related solutions and get familiarized with software development processes.	Apply
CO3	Perform requirement analysis, identify design methodologies and develop adaptable & reusable solutions of minimal complexity by using modern tools & advanced programming techniques.	Apply
CO4	Prepare technical report and deliver presentation.	Apply
CO5	Apply engineering and management principles to achieve the goal of the project.	Apply

MAPPING COURSE OUTCOMES (COs) – PROGRAM OUTCOMES (POs) AND COURSE OUTCOMES (COs) – PROGRAM SPECIFIC OUTCOMES (PSOs)

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	3	3	3	3	2	2	3	2	2	2	3	2	2	2
CO2	3	3	3	3	3	-	3	2	3	2	3	2	2	2
CO3	3	3	3	3	3	2	3	2	2	2	3	-	-	2
CO4	2	3	2	2	2	-	3	3	3	2	3	2	2	2
CO5	3	3	3	2	2	2	3	2	-	2	3	2	2	2

JUSTIFICATIONS FOR CO-PO MAPPING

MAPPING	LOW/MEDIUM / HIGH	JUSTIFICATION
102003/CS622T.CO1-PO1	HIGH	Identify technically and economically feasible problems by applying the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
102003/CS622T.CO1-PO2	HIGH	Identify technically and economically feasible problems by analysing complex engineering problems reaching substantiated conclusions using first principles of mathematics.
102003/CS622T.CO1-PO3	HIGH	Design solutions for complex engineering problems by identifying technically and economically feasible problems

MAPPING	LOW/MEDIUM /HIGH	JUSTIFICATION
102003/CS622T.CO1-PO4	HIGH	Identify technically and economically feasible problems by analysis and interpretation of data
102003/CS622T.CO1-PO5	MEDIUM	Responsibilities relevant to the professional engineering practice by identifying the problem.
102003/CS622T.CO1-PO6	MEDIUM	Identify technically and economically feasible problems by understanding the impact of the professional engineering solutions
102003/CS622T.CO1-PO7	HIGH	Apply ethical principles and commit to professional ethics to identify technically and economically feasible problems
102003/CS622T.CO1-PO8	MEDIUM	Identify technically and economically feasible problems by working as a team
102003/CS622T.CO1-PO9	MEDIUM	Communicate effectively with the engineering community by identifing technically and economically feasible problems
102003/CS622T.CO1-P010	MEDIUM	Demonstrate knowledge and understanding of engineering and management principles by selecting the technically and economically feasible problems
102003/CS622T.CO1-PO11	HIGH	Identify technically and economically feasible problems for long term learning.
102003/CS622T.CO1-PSO1	MEDIUM	Ability to identify, analyze and design solutions to identify technically and economically feasible problems
102003/CS622T.CO1-PSO2	MEDIUM	By designing algorithms and applying standard practices in software project development and Identifying technically and economically feasible problems
102003/CS622T.CO1-PSO3	MEDIUM	fundamentals of computer science in competitive research can be applied to Identify technically and economically feasible problems
102003/CS622T.CO2-PO1	HIGH	Identify and survey the relevant by applying the knowledge of mathematics, science, engineering fundamentals,
102003/CS622T.CO2-PO2	HIGH	Identify, formulate, review research literature, and analyze complex engineering problems get familiarized with software development processes
102003/CS622T.CO2-PO3	HIGH	Design solutions for complex engineering problems and design based on the relevant literature
102003/CS622T.CO2-PO4	HIGH	Use research-based knowledge including design of experiments based on relevant literature
102003/CS622T.CO2-PO5	HIGH	Identify and survey the relevant literature for getting exposed to related solutions and get familiarized with software development processes by using modern tools.
102003/CS622T.CO2-PO6	MEDIUM	Create, select, and apply appropriate techniques, resources, by identifying and surveying the relevant literature

MAPPING	LOW/MEDIUM /HIGH	JUSTIFICATION
102003/CS622T.CO2-PO7	HIGH	Apply ethical principles and commit to professional ethics based on the relevant literature
102003/CS622T.CO2-PO8	MEDIUM	Identify and survey the relevant literature as a team
102003/CS622T.CO2-PO9	HIGH	Identify and survey the relevant literature for a good communication to the engineering fraternity.
102003/CS622T.CO2-PO10	MEDIUM	Identify and survey the relevant literature to demonstrate knowledge and understanding of engineering and management principles
102003/CS622T.CO2-PO11	HIGH	Identify and survey the relevant literature for independent and lifelong learning
102003/CS622T.CO2-PSO1	MEDIUM	Design solutions for complex engineering problems by Identifying and survey the relevant literature
102003/CS622T.CO2-PSO2	MEDIUM	Identify and survey the relevant literature for acquiring programming efficiency by designing algorithms and applying standard practices
102003/CS622T.CO2-PSO3	MEDIUM	Identify and survey the relevant literature to apply the fundamentals of computer science in competitive research
102003/CS622T.CO3-PO1	HIGH	Perform requirement analysis, identify design methodologies by using modern tools & advanced programming techniques and by applying the knowledge of mathematics, science, engineering fundamentals.
102003/CS622T.CO3-PO2	HIGH	Identify, formulate, review research literature for requirement analysis, identify design methodologies and develop adaptable & reusable solutions
102003/CS622T.CO3-PO3	HIGH	Design solutions for complex engineering problems and perform requirement analysis, identify design methodologies.
102003/CS622T.CO3-PO4	HIGH	Use research-based knowledge including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions
102003/CS622T.CO3-PO5	HIGH	Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools
102003/CS622T.CO3-PO5	MEDIUM	Perform requirement analysis, identify design methodologies and assess societal, health, safety, legal, and cultural issues.
102003/CS622T.CO3-PO6	MEDIUM	Understand the impact of the professional engineering solutions in societal and environmental contexts and Perform requirement analysis, identify design methodologies and develop adaptable & reusable solutions
102003/CS622T.CO3-PO7	HIGH	Perform requirement analysis, identify design methodologies and develop adaptable & reusable solutions by applying ethical principles and commit to professional ethics
102003/CS622T.CO3-PO8	MEDIUM	Function effectively as an individual, and as a member or leader in teams, and in multidisciplinary settings

MAPPING	LOW/MEDIUM /HIGH	JUSTIFICATION
102003/CS622T.CO3-PO9	MEDIUM	Communicate effectively with the engineering community and with society at large to perform requirement analysis, identify design methodologies
102003/CS622T.CO3-PO10	MEDIUM	Demonstrate knowledge and understanding of engineering requirement analysis by identifying design methodologies
102003/CS622T.CO3-PO11	HIGH	Recognize the need for, and have the preparation and ability to engage in independent and lifelong learning in the broadest context of technological change by analysis, identify design methodologies and develop adaptable & reusable solutions.
102003/CS622T.CO3-PSO3	MEDIUM	The ability to apply the fundamentals of computer science in competitive research and prior to that perform requirement analysis, identify design methodologies
102003/CS622T.CO4-PO1	MEDIUM	Prepare technical report and deliver presentation by applying the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
102003/CS622T.CO4-PO2	HIGH	Identify, formulate, review research literature, and analyze complex engineering problems by preparing technical report and deliver presentation.
102003/CS622T.CO4-PO3	MEDIUM	Prepare Design solutions for complex engineering problems and create technical report and deliver presentation
102003/CS622T.CO4-PO4	MEDIUM	Use research-based knowledge including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions and prepare technical report and deliver presentation
102003/CS622T.CO4-PO5	MEDIUM	Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools and Prepare technical report and deliver presentation
102003/CS622T.CO4-PO7	HIGH	Prepare technical report and deliver presentation by applying ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice
102003/CS622T.CO4-PO8	HIGH	Prepare technical report and deliver presentation effectively as an individual, and as a member or leader in teams, and in multidisciplinary settings
102003/CS622T.CO4-PO9	HIGH	Communicate effectively with the engineering community and with society at large by prepare technical report and deliver presentation.
102003/CS622T.CO4-PO10	MEDIUM	Demonstrate knowledge and understanding of engineering and management principles and apply these to one's own work by prepare technical report and deliver presentation.

MAPPING	LOW/MEDIUM /HIGH	JUSTIFICATION
102003/CS622T.CO4-PO11	HIGH	Recognize the need for, and have the preparation and ability to engage in independent and lifelong learning in the broadest context of technological change by prepare technical report and deliver presentation
102003/CS622T.CO4-PSO1	MEDIUM	Prepare a technical report and deliver presentation to identify, analyze and design solutions for complex engineering problems in multidisciplinary areas.
102003/CS622T.CO4-PSO2	MEDIUM	To acquire programming efficiency by designing algorithms and applying standard practices in software project development and to prepare technical report and deliver presentation
102003/CS622T.CO4-PSO3	MEDIUM	To apply the fundamentals of computer science in competitive research and to develop innovative products to meet the societal needs by preparing technical report and deliver presentation
102003/CS622T.CO5-PO1	HIGH	Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
102003/CS622T.CO5-PO2	HIGH	Identify, formulate, review research literature, and analyze complex engineering problems by applying engineering and management principles to achieve the goal of the project.
102003/CS622T.CO5-PO3	HIGH	Apply engineering and management principles to achieve the goal of the project and to design solutions for complex engineering problems and design system components or processes that meet the specified needs
102003/CS622T.CO5-PO4	MEDIUM	Apply engineering and management principles to achieve the goal of the project and use research-based knowledge including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions
102003/CS622T.CO5-PO5	MEDIUM	Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools and to apply engineering and management principles to achieve the goal of the project
102003/CS622T.CO5-PO6	MEDIUM	Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal, and cultural issues and the consequent responsibilities by applying engineering and management principles to achieve the goal of the project
102003/CS622T.CO5-PO7	MEDIUM	Understand the impact of the professional engineering solutions in societal and environmental contexts, and apply engineering and management principles to achieve the goal of the project
102003/CS622T.CO5-PO8	HIGH	Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice and to use the engineering and management principles to achieve the goal of the project

MAPPING	LOW/MEDIUM /HIGH	JUSTIFICATION
102003/CS622T.CO5-PO9	MEDIUM	Function effectively as an individual, and as a member or leader in teams, and in multidisciplinary settings and to apply engineering and management principles to achieve the goal of the project
102003/CS622T.CO5-PO11	MEDIUM	Demonstrate knowledge and understanding of engineering and management principles and apply these to one's own work, as a member and leader in a team. Manage projects in multidisciplinary environments and to apply engineering and management principles to achieve the goal of the project
102003/CS622T.CO5-PO11	HIGH	Recognize the need for, and have the preparation and ability to engage in independent and lifelong learning in the broadest context of technological change and to apply engineering and management principles to achieve the goal of the project
102003/CS622T.CO5-PSO1	MEDIUM	The ability to identify, analyze and design solutions for complex engineering problems in multidisciplinary areas Apply engineering and management principles to achieve the goal of the project
102003/CS622T.CO5-PSO2	MEDIUM	The ability to acquire programming efficiency by designing algorithms and applying standard practices in software project development to deliver quality software products meeting the demands of the industry and to apply engineering and management principles to achieve the goal of the project
102003/CS622T.CO5-PSO3	MEDIUM	The ability to apply the fundamentals of computer science in competitive research and to develop innovative products to meet the societal needs thereby evolving as an eminent researcher and entrepreneur and apply engineering and management principles to achieve the goal of the project